

Índice

1. Introducción	3
2. Escenarios	3
3. Vista Lógica	4
3.1. Grafo Acíclico Dirigido (DAG)	4
3.1.1. Pipeline de filtrado	4
3.1.2. Q1 — Cuentas con transacciones en USD menores a 50	4
3.1.3. Q2 — Máxima transacción en USD por banco	5
3.1.4. Q3 — Transacciones por debajo de la centésima parte del promedio	5
3.1.5. Q4 — Pares de cuentas con cinco intermediarios	6
3.1.6. Q5 — Conteo de microtransacciones Wire/ACH	6
4. Vista de Procesos	7
4.1. Diagrama de Secuencia	7
4.2. Diagramas de Actividad	8
5. Manejo de EOF	17
5.1. Topología de anillo	17
5.2. Garantía de entrega confiable	17
5.3. Trade-offs	18
6. Vista Física y de Despliegue	18
6.1. Tipos de nodos lógicos	18
6.1.1. Actores	18
6.1.2. Workers	19
6.2. Diagrama de Robustez	19
6.3. Diagrama de Despliegue	27
7. Vista de Desarrollo	28
7.1. Diagrama de Paquetes	28
8. Planificación y Organización	29
8.1. Listado de Tareas a Ejecutar	29
8.2. División del Trabajo entre Integrantes	30

1. Introducción

El presente documento detalla la arquitectura y el diseño de un sistema distribuido cuyo propósito es analizar extractos de transacciones entre cuentas bancarias para detectar anomalías. Este análisis se enmarca en la detección del lavado de activos, un delito que consiste en cambiar bienes o dinero obtenidos mediante actos ilícitos por dinero legítimo. Para lograr esto, los infractores utilizan la red digital de pagos aplicando diversas estrategias de ofuscación, tales como transacciones Fan-out, Fan-in, patrones Scatter-Gather, grafos Bipartitos y Stack.

Para hacer frente a esta problemática, se diseñó una solución optimizada para entornos multi-computadoras. La arquitectura propuesta permite escalar los elementos de cómputo horizontalmente para soportar el incremento en los volúmenes de información y clientes simultáneos a procesar. A nivel de infraestructura, el sistema se apoya en un Middleware que abstrae la comunicación basada en grupos. Asimismo, el flujo de procesamiento soporta una única ejecución continua y provee mecanismos de cierre ordenado.

Los datos utilizados para el procesamiento provienen del dataset de simulaciones bancarias de IBM (disponible en Kaggle.), integrándose además con la API Frankfurter para obtener las cotizaciones diarias necesarias para las conversiones de divisas a USD.

2. Escenarios

Query 1 — Micropagos (Fan-out / Fan-in): Se buscan todas las transacciones en dólares estadounidenses (USD) cuyo monto sea estrictamente menor a \$50. Para cada una se reportan la cuenta de origen, la cuenta de destino y el monto transferido. Montos pequeños y frecuentes son un indicador típico de los patrones *fan-out* y *fan-in*, donde una cuenta dispersa o concentra fondos en pequeñas fracciones para evitar los umbrales de reporte automático de las entidades bancarias.

Query 2 — Máximos por Entidad Bancaria: Para cada banco de origen presente en el conjunto de transacciones en USD, se identifica la transferencia de mayor monto. El resultado incluye el nombre del banco, la cuenta emisora y el monto correspondiente. Esta consulta permite detectar cuentas que concentran movimientos de alto valor dentro de una misma entidad, lo cual puede ser indicativo de operaciones atípicas o de una cuenta central en un esquema de lavado.

Query 3 — Anomalías Históricas (Stack): Se analiza el período [2022-09-06, 2022-09-15] comparándolo con el período histórico de referencia [2022-09-01, 2022-09-05]. Para cada formato de pago, se calcula el promedio de montos en USD del período de referencia. Luego se reportan las transacciones del período analizado cuyo monto sea inferior a un centésimo (1 %) de dicho promedio. Este tipo de anomalía corresponde al patrón *stack*: transferencias acumuladas de montos ínfimos a lo largo del tiempo, diseñadas para pasar desapercibidas.

Query 4 — Patrón Scatter-Gather: Se identifican las cuentas que participan en un esquema de *scatter-gather* con exactamente una cuenta intermediaria. La condición es que la cuenta de origen haya realizado transferencias en USD hacia al menos 5 cuentas intermediarias distintas durante el período [2022-09-01, 2022-09-05], y que cada una de esas cuentas intermediarias haya transferido fondos hacia una misma cuenta de destino final. Este patrón describe el flujo *fan-out* desde un origen, seguido de un *fan-in* hacia un destino, utilizando cuentas intermediarias.

Query 5 — Micropagos Internacionales (Wire / ACH): Se contabilizan las transacciones del período [2022-09-01, 2022-09-05] realizadas mediante los formatos de pago *Wire* o *ACH* cuyo monto, una vez convertido a USD utilizando el tipo de cambio diario provisto por la API de Frankfurter, sea estrictamente menor a \$1. La conversión dinámica permite detectar micropagos internacionales que, expresados en moneda local, pueden no parecer sospechosos pero que en USD revelan un volumen anómalo de operaciones de bajísimo valor.

3. Vista Lógica

En esta sección presentamos la vista lógica del sistema, cuyo objetivo es describir el flujo de datos desde la fuente de datos enviado por el cliente hasta el sumidero final con los resultados consolidados que se devuelven al mismo cliente. El énfasis está puesto en las transformaciones que sufre cada tupla, sin todavía comprometernos con cómo se distribuyen físicamente esas transformaciones entre nodos.

3.1. Grafo Acíclico Dirigido (DAG)

Representamos el flujo lógico de los datos mediante un Grafo Acíclico Dirigido (DAG). Cada nodo es una transformación lógica (**source**, **router**, **map**, **filter**, **reducer**, **aggregator**, **joiner** o **sink**) y cada arista lleva la tupla concreta que se transmite entre etapas a través de un *exchange* o *queue* de RabbitMQ.

Este esquema de parciales y combinación es posible porque las agregaciones involucradas (máximos, sumas, conteos) dan el mismo resultado tanto si se computan de una sola vez como por partes. Cuando la operación no es directamente combinable (como el promedio de Q3, dado que el promedio de promedios no equivale al promedio real) se acumulan en su lugar magnitudes que sí lo son (**suma** y **conteo**) y la operación final se aplica una única vez sobre el agregado.

Para evitar un único diagrama sobrecargado, descomponemos el grafo en un **pipeline de filtrado**, común a todas las consultas, y **cinco sub-DAG independientes**, uno por consulta. Todos los sub-DAG parten de las *transacciones filtradas* que produce el pipeline inicial y desembocan en sus respectivos resultados.

3.1.1. Pipeline de filtrado

La fuente **Transacciones** produce una tupla por cada transacción con todos sus campos (**TxID**, **FromBank**, **FromAcc**, **ToBank**, **ToAcc**, **PaymentCurrency**, **Amount**, **PaymentFormat**, **Date**). Esa tupla atraviesa una cadena de filtros que actúa como *smart router*: evalúa predicados sobre moneda, período, monto y medio de pago, y encamina cada transacción hacia las colas terminales que alimentan cada consulta. La notación **TRANSACCIÓN*** denota la tupla ya filtrada y proyectada que las ramas Q1–Q5 reciben como entrada.

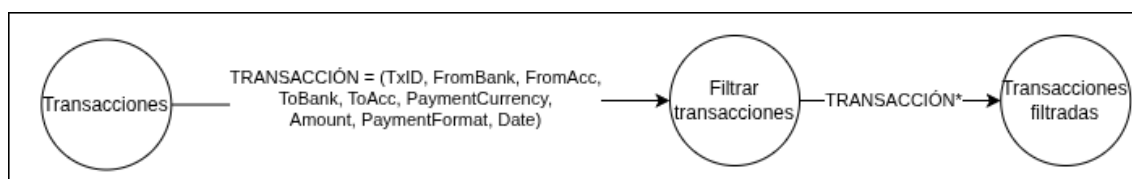


Figura 1: Pipeline base de filtrado, común a todas las consultas.

3.1.2. Q1 — Cuentas con transacciones en USD menores a 50

Es la rama más simple, donde cada tupla se procesa de forma independiente. Sobre las transacciones en USD con monto inferior a 50, se proyecta cada una a (**FromAcc**, **ToAcc**, **Amount**) y se emite directamente al resultado.

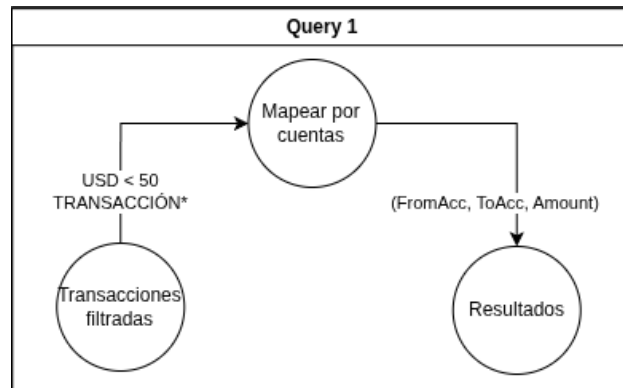


Figura 2: Sub-DAG de la consulta Q1.

3.1.3. Q2 — Máxima transacción en USD por banco

El máximo se calcula en dos etapas. Las transacciones en USD se reparten por banco de origen y, dentro de cada partición, se obtiene la máxima transacción, produciendo un máximo parcial (*FromBank*, *FromAcc*, *MaxAmount*). Esos parciales se combinan luego en el máximo global de cada banco. En paralelo, el cliente envía el catálogo de bancos (*BankID*, *BankName*), que se cruza con cada máximo para incorporar el nombre del banco, produciendo (*BankID*, *BankName*, *FromBank*, *FromAcc*, *MaxAmount*).

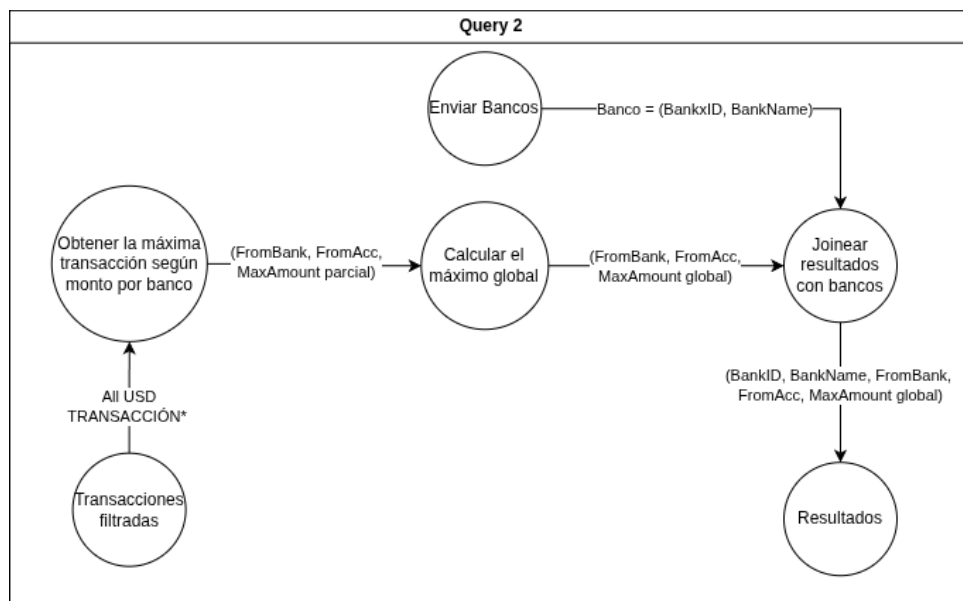


Figura 3: Sub-DAG de la consulta Q2.

3.1.4. Q3 — Transacciones por debajo de la centésima parte del promedio

Q3 ejecuta dos sub-ramas en paralelo. Sobre el período de referencia [2022-09-01, 2022-09-05], las transacciones se agrupan por *PaymentFormat* para acumular (*suma*, *conteo*) parciales que luego se combinan, obteniendo el promedio de cada medio de pago. La otra sub-rama retiene las transacciones del período analizado [2022-09-06, 2022-09-15] a la espera de ese promedio. Una vez disponible, se conservan únicamente las transacciones cuyo monto es inferior a la centésima parte del promedio de su *PaymentFormat*, emitiendo (*FromAcc*, *PaymentFormat*, *Amount*).

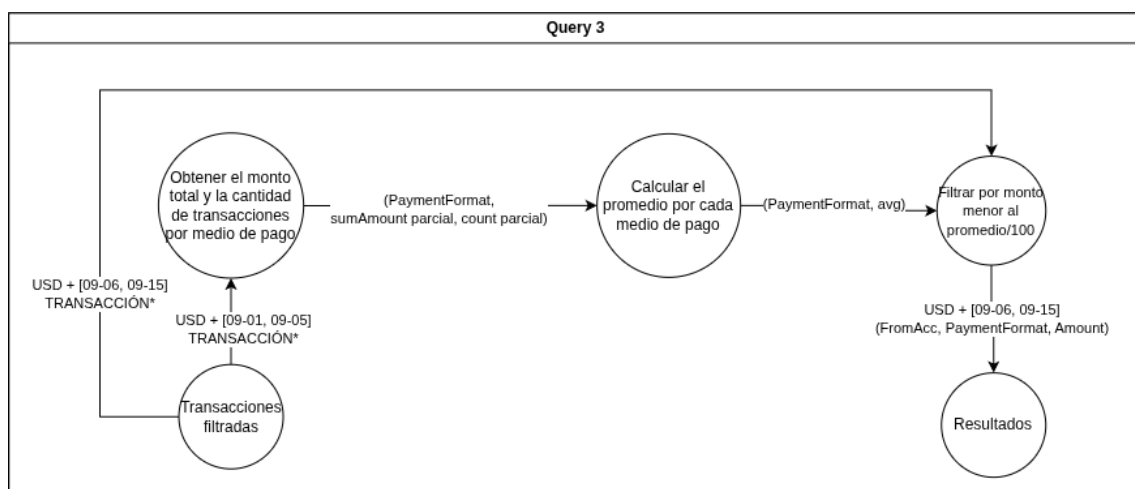


Figura 4: Sub-DAG de la consulta Q3.

3.1.5. Q4 — Pares de cuentas con cinco intermediarios

Para detectar cadenas de intermediarios, cada transacción en USD del período [2022-09-01, 2022-09-05] se duplica y se reparte por su cuenta de origen y por su cuenta de destino (Acc, OtherAcc, ShardingAcc), agrupando las que comparten una cuenta. Antes de la etapa más costosa se aplica un **pre-filtrado**, donde se conservan únicamente las cuentas con cinco o más contrapartes distintas —es decir, con al menos cinco puntos de entrada o de salida— y se descarta el resto, que no puede formar parte del patrón. Esto reduce drásticamente el volumen que llega a la reconstrucción de caminos. Sobre las transacciones que sobreviven se arman los caminos de tres cuentas (FromAcc, IntermediateAcc, ToAcc) y, combinando los parciales, se retienen los pares (origen, destino) conectados a través de al menos cinco cuentas intermedias distintas, emitiendo (FromAcc, ToAcc).

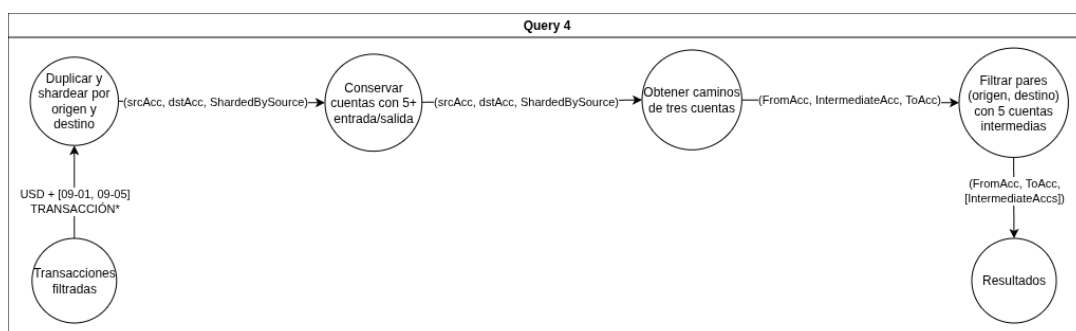


Figura 5: Sub-DAG de la consulta Q4.

3.1.6. Q5 — Conteo de microtransacciones Wire/ACH

Sobre las transacciones de tipo Wire/ACH del período [2022-09-01, 2022-09-05], se convierte el monto a USD y se descartan las conversiones iguales o superiores a 1 USD. El conteo se realiza en dos etapas, donde cada partición cuenta localmente sus transacciones (partial_count) y esos conteos parciales se suman en el total global (total_count).

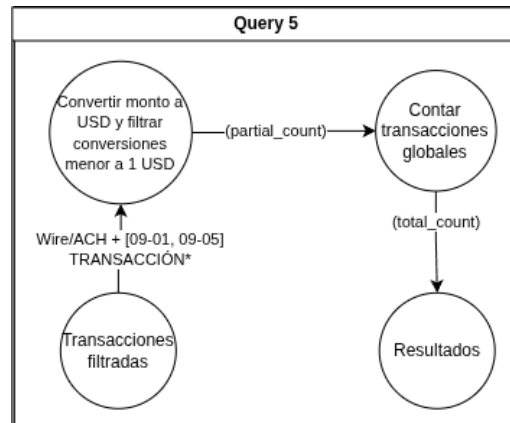


Figura 6: Sub-DAG de la consulta Q5.

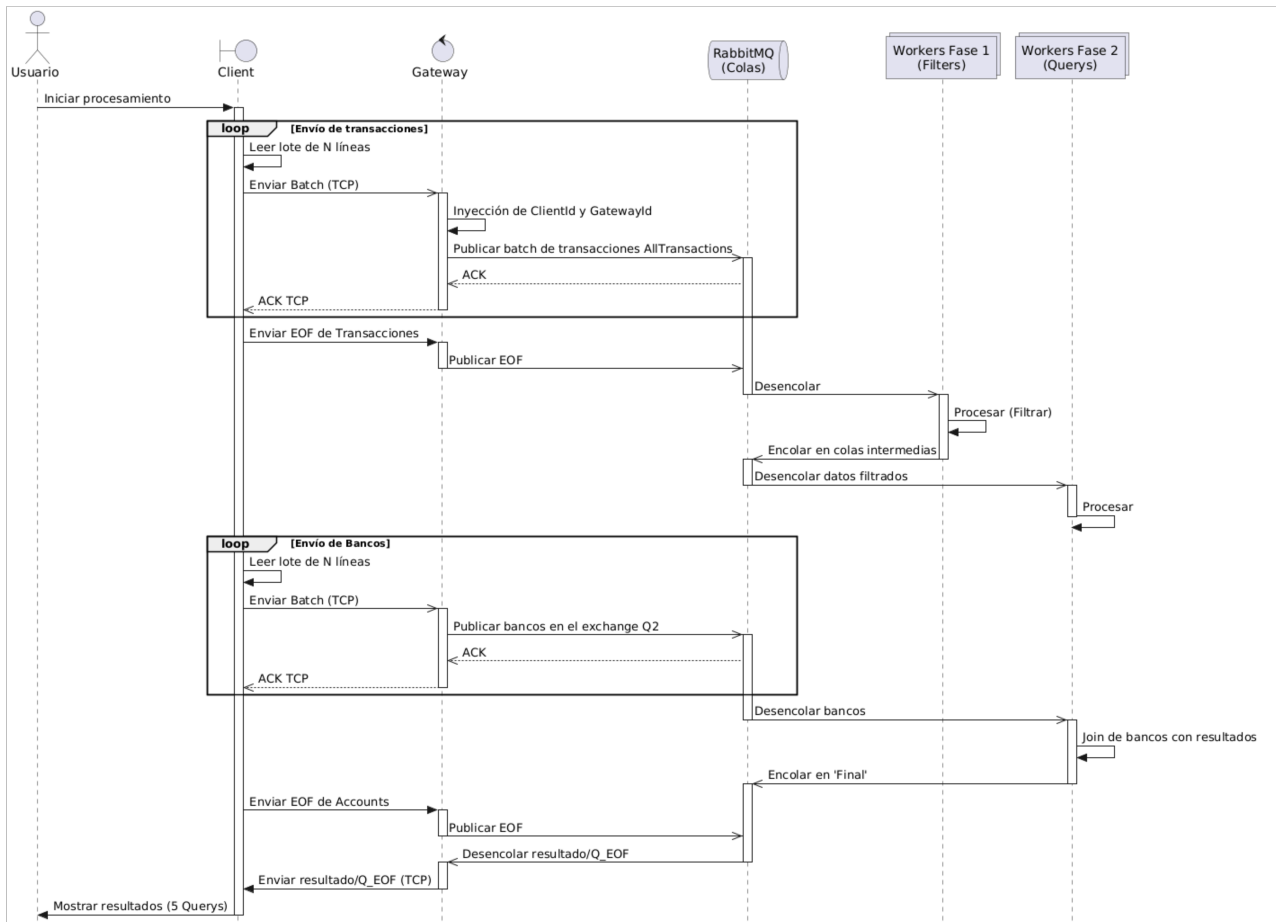
Todas las ramas convergen en el *Final Joiner*, que envía el resultado parcial de cada cliente por cada consulta hacia el Gateway. Este la reenvía al cliente por el mismo socket TCP por el que llegaron las transacciones.

4. Vista de Procesos

En la presente sección describimos la dinámica del sistema, es decir, cómo interactúan los distintos componentes a lo largo de la ejecución y cómo se coordinan para producir los resultados de cada consulta. Presentamos en primer lugar un diagrama de secuencia que ilustra las interacciones entre las entidades principales, y a continuación los diagramas de actividad que detallan el comportamiento interno de cada componente.

4.1. Diagrama de Secuencia

El diagrama que sigue resume el flujo completo de una sesión de cliente: el establecimiento de la conexión TCP, el envío de *batches* de transacciones y bancos, la propagación de éstas hacia las distintas *Queues*, el procesamiento por las consultas Q1–Q5, la convergencia en el *Final Joiner* y la entrega final de resultados al cliente.

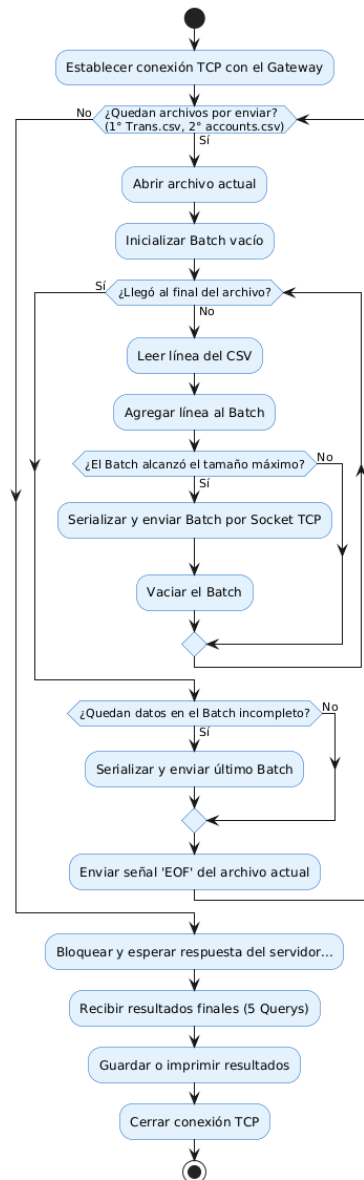


Resulta importante destacar que la comunicación entre cliente y *gateway* es síncrona y orientada a la conexión (TCP/IP), mientras que toda la comunicación interna entre el *gateway* y los *workers* es asíncrona y mediada por RabbitMQ. Esta separación permite desacoplar la velocidad de ingreso de los datos del cliente de la velocidad de procesamiento del *backend*, y aislar al cliente de los detalles de coordinación interna del sistema.

4.2. Diagramas de Actividad

A continuación se presentan los diagramas de actividad de los distintos componentes del sistema. Cada uno describe el procesamiento que un *worker* realiza por cada mensaje recibido, así como su comportamiento al detectar la señal de fin de procesamiento (EOF). El protocolo concreto mediante el cual se garantiza que el EOF refleja el cierre de los datos del cliente se describe en la sección *Manejo de EOF*; en lo que sigue asumimos que esa garantía existe.

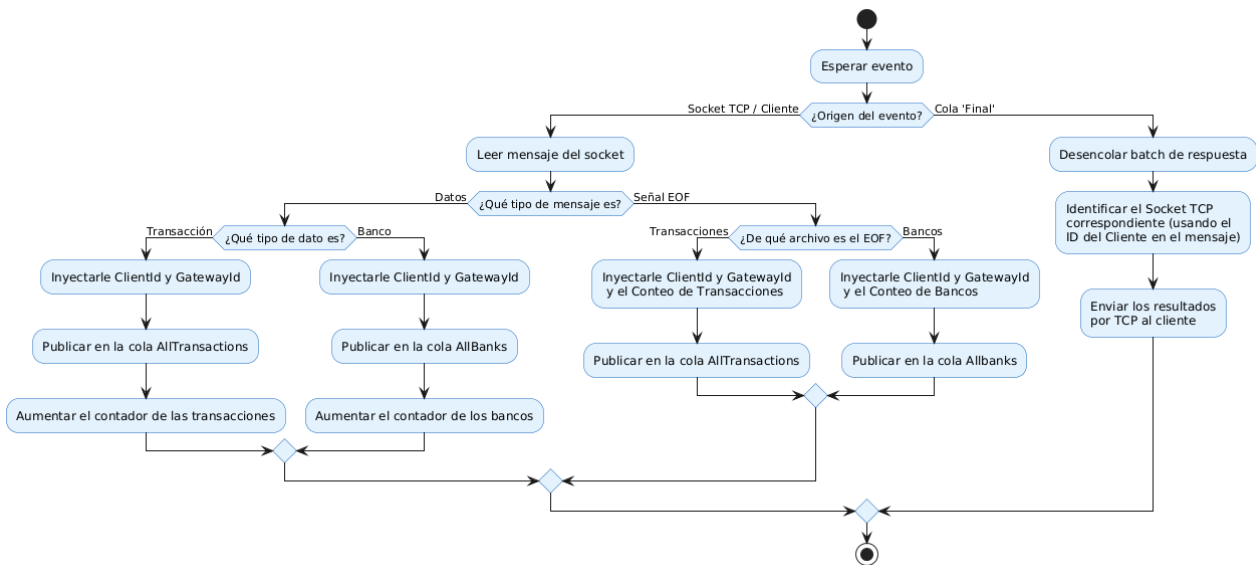
- **Cliente.** El cliente selecciona aleatoriamente uno de los *gateways* disponibles e intenta establecer una conexión TCP. En caso de fallo, reintenta con backoff exponencial, rotando entre los gateways disponibles hasta establecer la conexión. Una vez conectado, lee el archivo *Trans.csv* línea por línea y arma *batches* de tamaño configurable que envía por el socket. Al alcanzar el final del archivo serializa el último *batch* y emite la señal EOF sobre el mismo socket. Una vez hecho esto, sigue exactamente los mismos pasos para el archivo *accounts.csv*. A continuación se bloquea a la espera de los resultados que son recibidos y guardados en disco progresivamente. Al llegar los 5 EOF (1 por query) el cliente se cierra.



- **Gateway.** El gateway actúa como proxy entre los clientes TCP y RabbitMQ. Al recibir una nueva conexión, realiza un *handshake* con el cliente asignándole un **ClientID** UUID único. A partir de ese momento maneja dos flujos en paralelo por cliente.

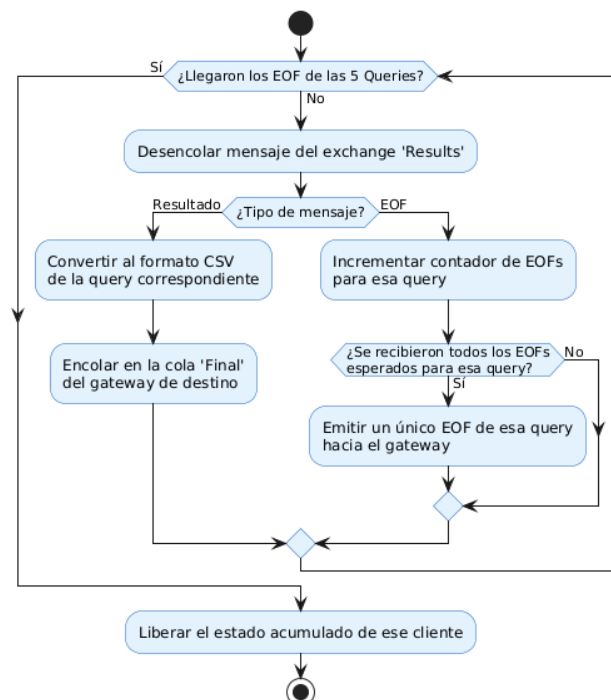
En el flujo de **entrada**, por cada *batch* de transacciones o cuentas recibido por TCP lo etiqueta con el **ClientID** y **GatewayID** correspondiente y lo publica en la cola transacciones o cuentas respectivamente, confirmando la recepción al cliente con un **ACK**. Lo mismo ocurre con los mensajes EOF de cada archivo.

Además en el flujo de **salida**, se consume continuamente de la cola **final_{gatewayID}**. Cada resultado llega con **ClientID** y **GatewayID** embebidos; el gateway identifica el cliente destinatario en su registro y le reenvía los resultados por TCP. Utiliza un esquema *at-least-once*: el ACK de AMQP se emite recién cuando el cliente confirma haber recibido el *batch*. La sesión se cierra al entregar los 5 EOF de queries esperados.

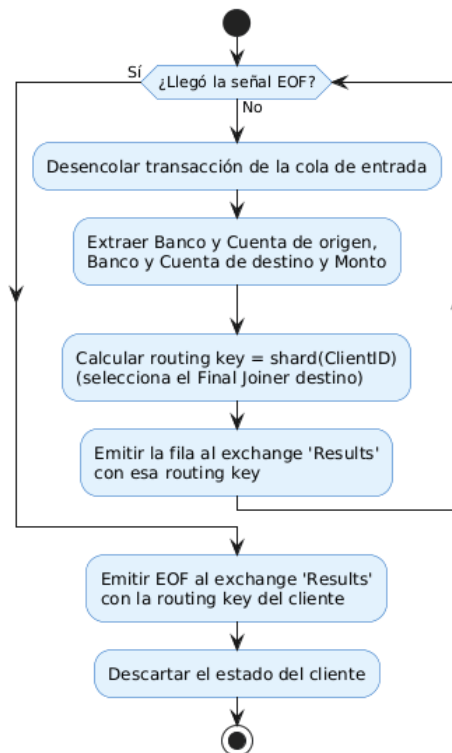


- **Final Joiner.** Consume del *exchange results* (shardeado: cada réplica recibe una fracción de los mensajes). Para cada mensaje entrante realiza dos tareas: convierte el formato binario interno al formato CSV de salida correspondiente a cada consulta (Q1–Q5), y lo encola en la cola *final_{gatewayID}* del gateway de origen, preservando el *ClientID* para que el gateway pueda rutear la respuesta al cliente correcto.

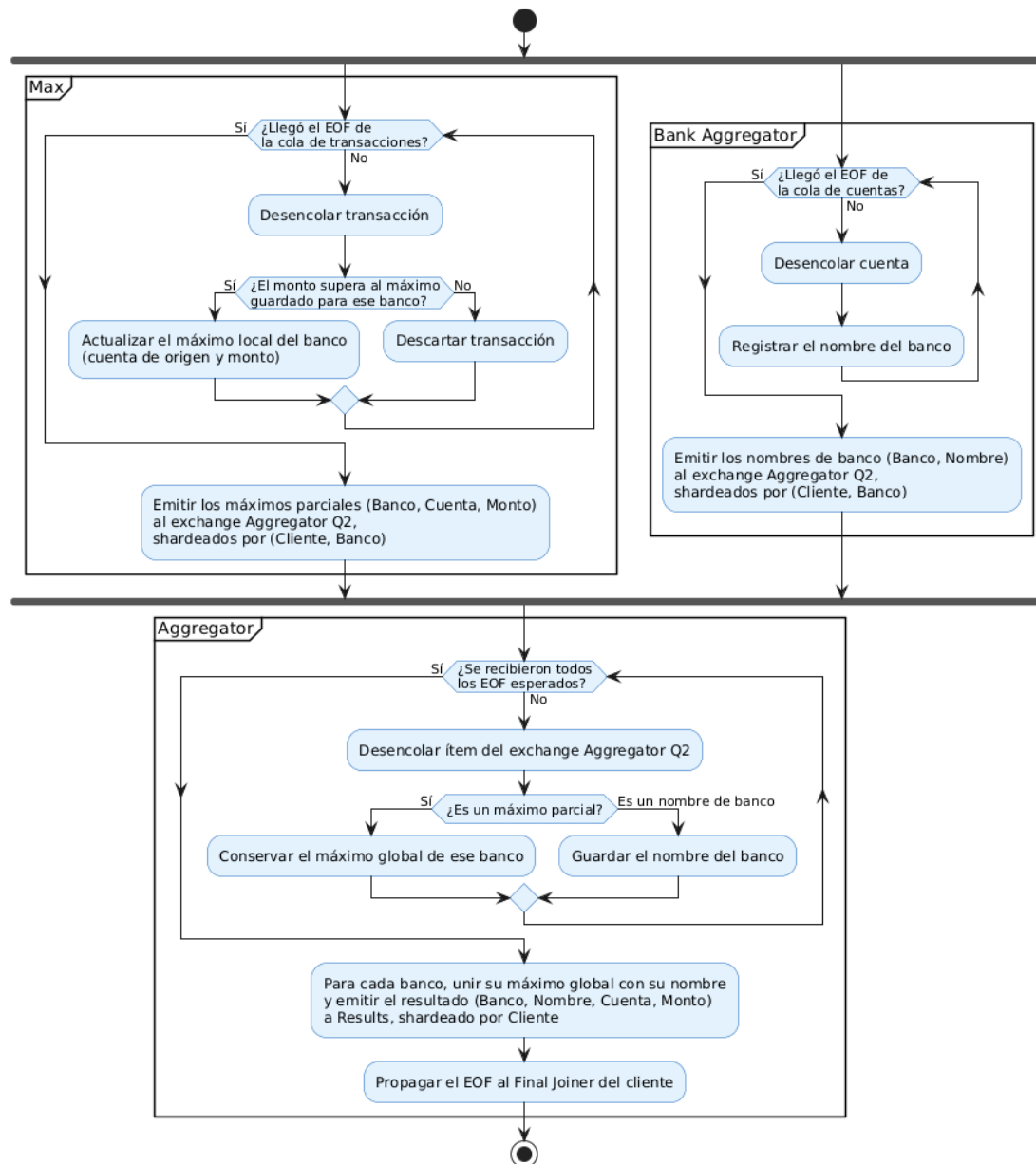
En cuanto al tratamiento de los EOF, cada consulta configurada mantiene un contador de los EOF que debe recibir. En particular, espera *EXPECTED EOFs_Q{n}* EOFs provenientes de upstream (emitidos por las réplicas terminales de cada rama) antes de propagar un único EOF downstream hacia el gateway. De esta manera, se abstrae la multiplicidad interna de réplicas y se preserva la interfaz esperada por el gateway y, en consecuencia, por el cliente al enviar un solo EOF por consulta.



- **Query 1 (Micropagos).** Esta rama tiene una sola etapa (el *Sharder Q1*), replicable en varias instancias, que opera de manera *online*. Por cada transacción extrae los datos relevantes (banco y cuenta de origen, banco y cuenta de destino, y monto) y la emite al *exchange Results* *shardeada* por *ClientID*, de modo que todos los resultados de un mismo cliente converjan en el *Final Joiner* que le corresponde. Al recibir el EOF lo reenvía a *Results* con la misma *routing key* del cliente y descarta su estado parcial.

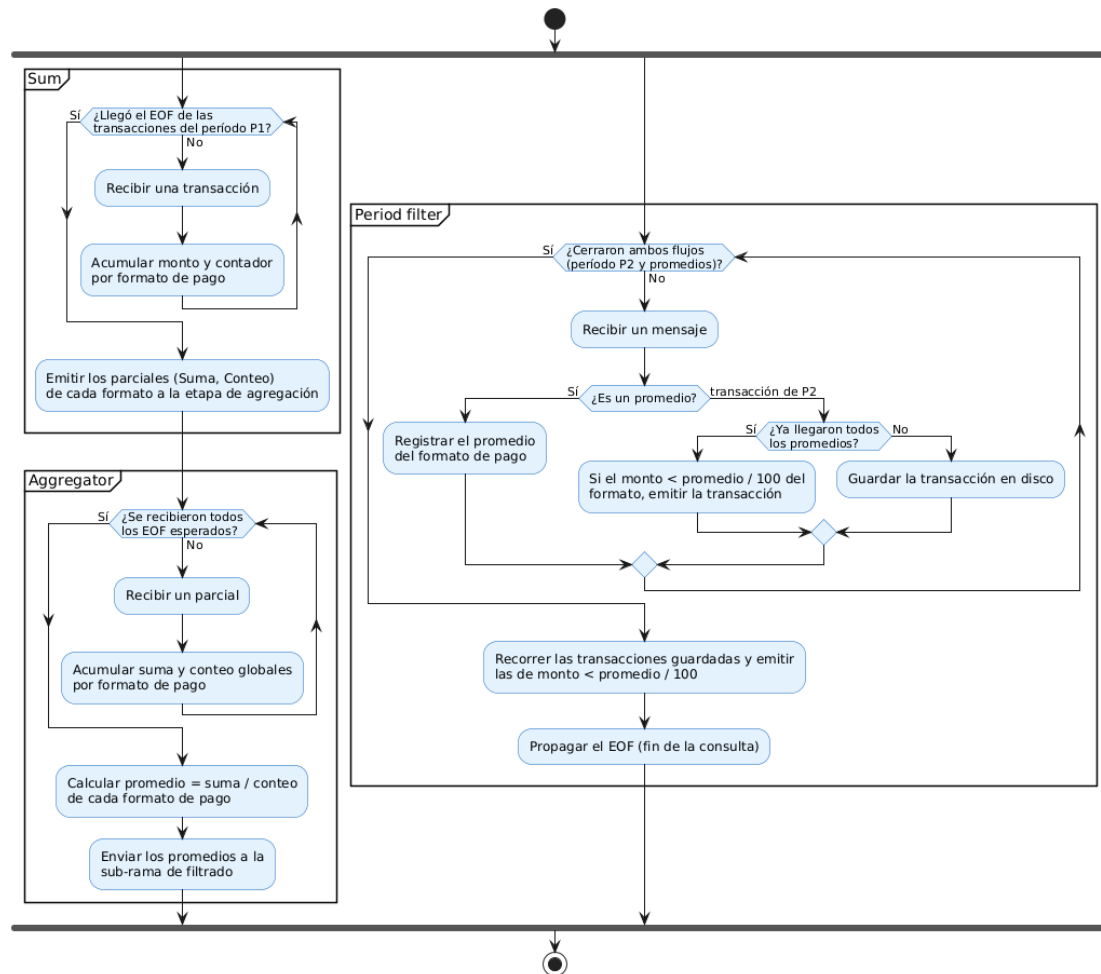


- **Query 2 (Máximos por Entidad Bancaria).** Se compone de tres etapas. La etapa *Max* mantiene un máximo local por banco mientras consume transacciones de su correspondiente cola, registrando la cuenta de origen y el monto de la mayor transacción de cada banco; al recibir el EOF emite sus máximos parciales (**Banco, Cuenta, Monto**) al *exchange Aggregator Q2*, *shardeados* por la clave (**Cliente, Banco**). En paralelo, la etapa *Bank Aggregator* consume el catálogo de cuentas y acumula el nombre de cada banco, que publica en el mismo *exchange* con idéntico *sharding*, de modo que el máximo de un banco y su nombre converjan en el mismo agregador. Finalmente, la etapa *Aggregator* consume ambos flujos: conserva el máximo global de cada banco y registra los nombres;. Cuando ha recibido todos los EOF esperados, junta cada máximo con el nombre de su banco y emite el resultado (**Banco, Nombre, Cuenta, Monto**) a *Results*, *shardeado* por cliente hacia su *Final Joiner*, junto con su EOF.

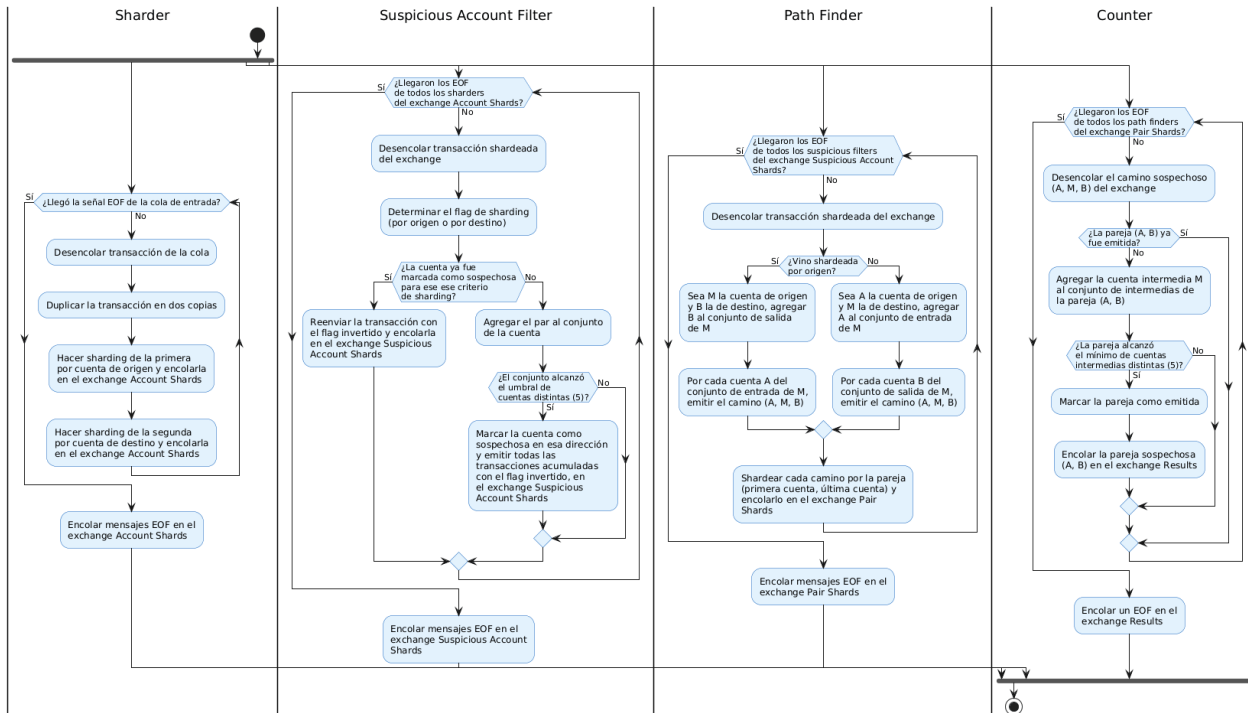


- **Query 3 (Anomalías Históricas).** Esta consulta se organiza en dos sub-ramas que operan concurrentemente. La primera sub-rama, encargada del cálculo de promedios, consta de dos etapas. En la etapa *Sum*, se procesan las transacciones correspondientes al período de referencia (P1), manteniendo para cada formato de pago una suma acumulada y un conteo parcial. Al recibir el EOF, estos valores parciales son enviados a la etapa de agregación. Luego, el *Aggregator* consolida los resultados recibidos, calcula el promedio asociado a cada formato de pago y lo distribuye hacia la sub-rama de filtrado.

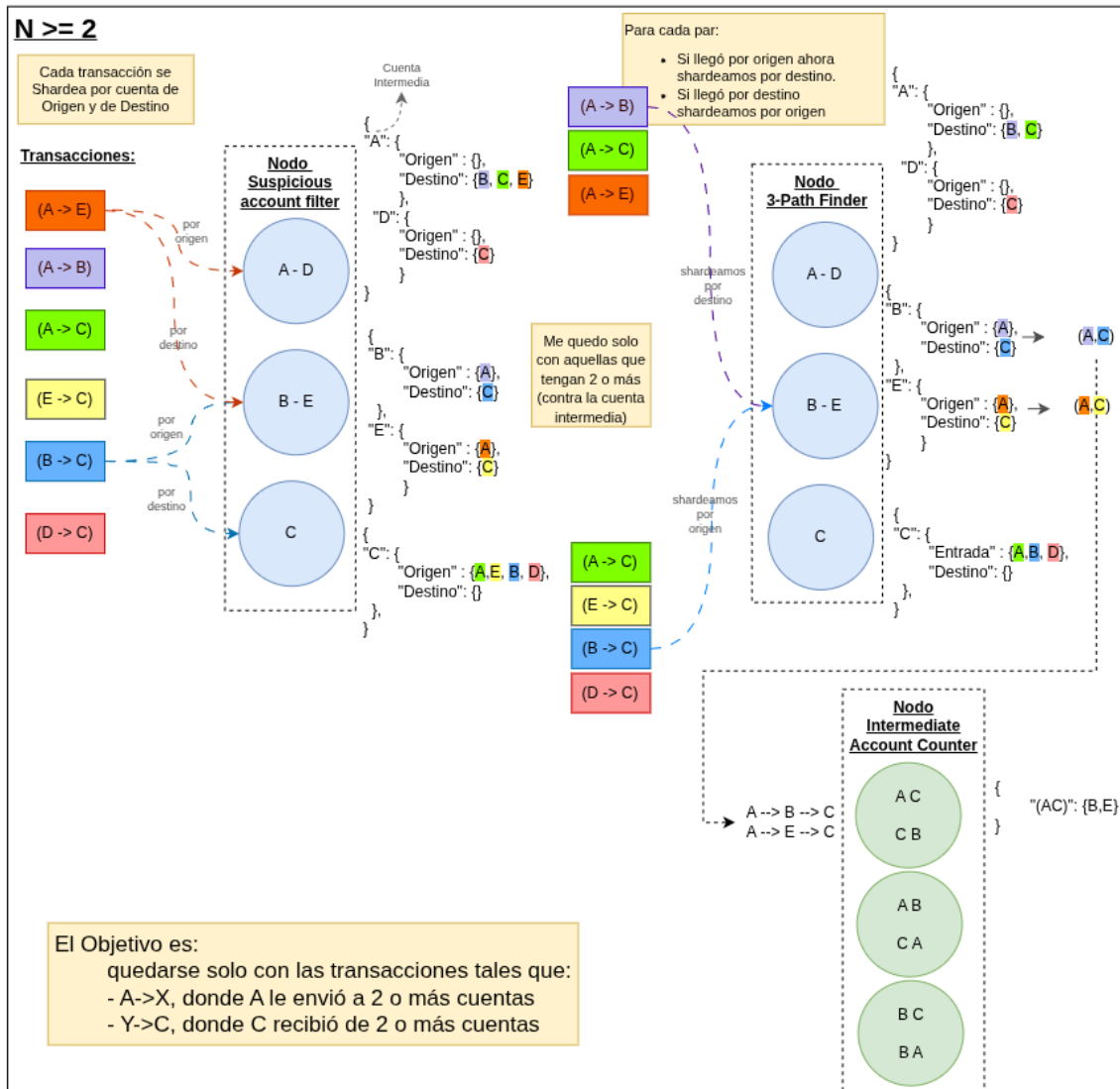
De manera simultánea, el *Period filter* consume dos flujos independientes: las transacciones del período analizado (P2) y los promedios calculados previamente. Mientras los promedios aún no están disponibles en su totalidad, las transacciones de P2 se persisten temporalmente en disco. Una vez completada la recepción de promedios, las nuevas transacciones se filtran a medida que arriban. Cuando ambos flujos finalizan, el componente recorre las transacciones almacenadas y emite como resultado únicamente aquellas cuyo monto es inferior a un centésimo del promedio correspondiente a su formato de pago. Finalmente, propaga el EOF asociado a la consulta.



- Query 4 (Scatter-Gather).** La rama está compuesta por cuatro etapas en *pipeline*. El *Sharder* recibe cada transacción y emite dos copias *shardeadas*: una marcada como 'shardeada por origen' y otra como 'shardeada por destino', ambas hacia el exchange *Account Shards*. El *Suspicious Account Filter* consume de dicho *exchange* y, por cada cuenta, acumula el conjunto de cuentas distintas con las que opera en cada dirección (entrada y salida); cuando una cuenta alcanza el umbral de cinco pares distintos en una dirección, la marca como sospechosa y emite hacia el exchange *Suspicious Account Shards* todas las transacciones acumuladas con el flag de *sharding* invertido (a partir de ese momento reenvía 1 a 1 las nuevas transacciones de esa cuenta). De esta forma solo avanzan las transacciones de cuentas con actividad suficiente. El *3-Path Finder* consume del exchange *Suspicious Account Shards* y, para cada cuenta intermedia, mantiene los conjuntos de cuentas que le envían fondos y a las que ella envía fondos; cada vez que descubre un nuevo camino sospechoso de la forma (origen, intermedia, destino), lo *shardea* por el par (origen, destino) y lo emite al exchange *Pair Shards*. Finalmente, el *Intermediate Account Counter* agrupa los caminos por par y emite a *Results* aquellos pares (origen, destino) que poseen al menos cinco cuentas intermedias distintas.



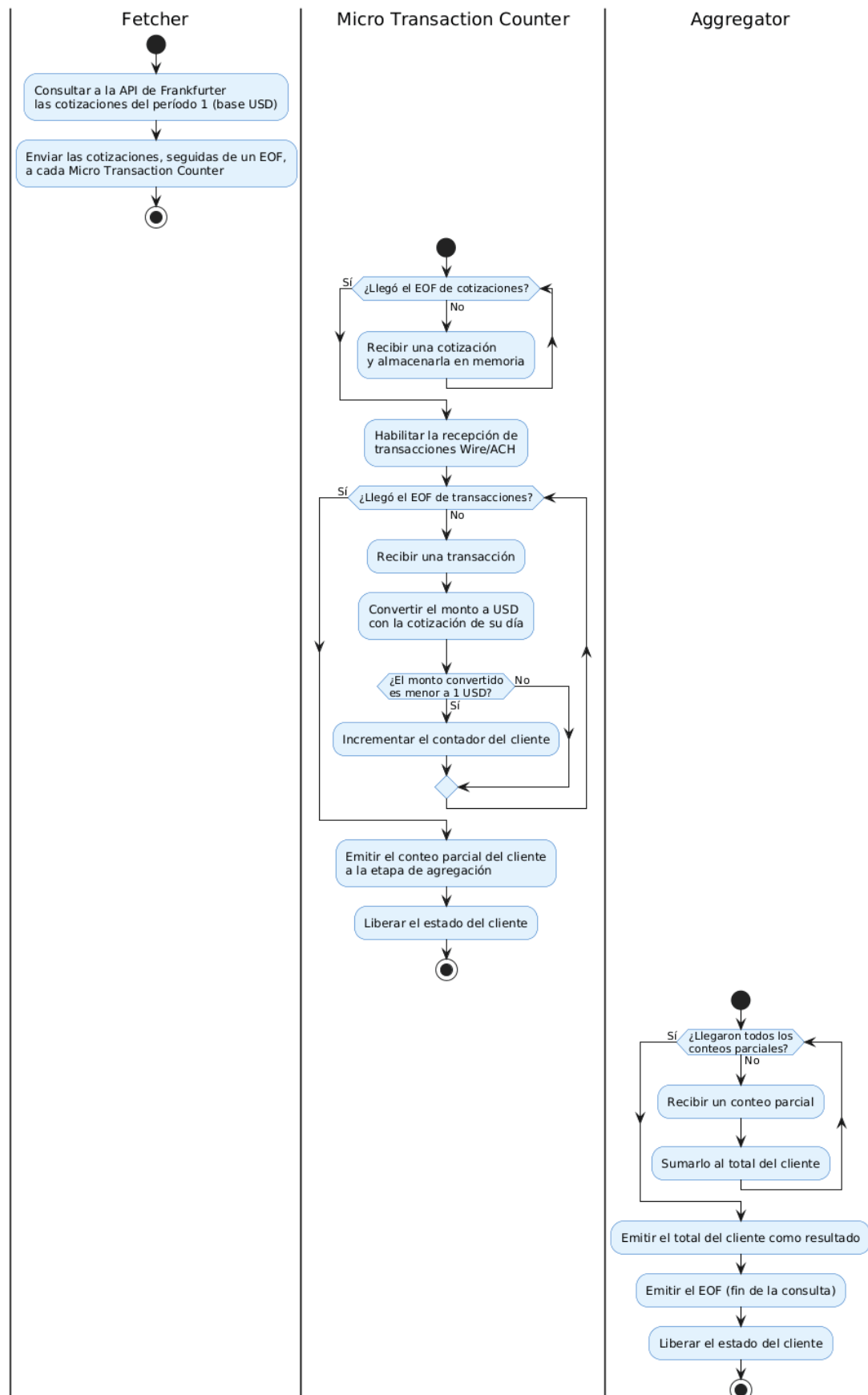
Para hacer tangible el algoritmo de Q4, el siguiente esquema muestra una ejecución de ejemplo. A la izquierda aparecen las transacciones de entrada, y en el centro la representación interna del *Suspicious Account Filter* y el *3-Path Finder* para cada cuenta intermedia: por cada cuenta se mantienen los conjuntos **Entrada** (cuentas que le envían fondos) y **Salida** (cuentas a las que envía fondos). En el *Suspicious Account Filter*, estos conjuntos se utilizan para filtrar solo a las cuentas con 5 o más cuentas a las que envió alguna transacción o aquellas que recibieron transacciones de 5 o más cuentas distintas. Por su parte, en el *3-Path Finder*, cada vez que la cuenta acumula un nuevo elemento en **Entrada** o en **Salida**, se emiten todos los nuevos caminos que se pueden armar con ese nuevo elemento. A la derecha se observa cómo el *Intermediate Account Counter* agrupa los caminos por el par (**origen, destino**) y mantiene el conjunto de cuentas intermediarias asociadas a cada par, emitiendo el par a **Results** en cuanto dicho conjunto alcanza las cinco cuentas intermedias distintas.



- **Query 5 (Micropagos Internacionales).** Esta consulta combina una etapa de inicialización de datos externos con dos etapas posteriores de procesamiento y agregación. En primer lugar, el componente *Fetcher* realiza una única consulta a la API de Frankfurter para obtener las cotizaciones correspondientes al período 1, utilizando USD como moneda base. Las cotizaciones recuperadas son distribuidas a todos los nodos *Micro Transaction Counter*, seguidas de un EOF que indica la finalización de la carga de referencias.

Cada *Micro Transaction Counter* almacena inicialmente las cotizaciones recibidas en memoria y recién comienza a procesar transacciones una vez recibido el EOF asociado a dicho flujo. A continuación, consume las transacciones de tipo Wire/ACH del período analizado y, para cada una, convierte el monto a USD utilizando la cotización correspondiente a la fecha de la operación. Si el valor convertido resulta inferior a 1 USD, incrementa un contador local asociado al cliente de la transacción.

Al finalizar el flujo de transacciones, cada nodo emite sus conteos parciales hacia la etapa de agregación. Finalmente, el *Aggregator* consolida los resultados provenientes de todos los *workers*, calcula el total de micropagos internacionales por cliente y publica el resultado final junto con el EOF de la consulta.



5. Manejo de EOF

Antes de avanzar con la vista física, resulta necesario describir el mecanismo mediante el cual el sistema garantiza el cierre ordenado del procesamiento de cada cliente. Dado que cualquier consulta involucra grupos de *workers* idénticos consumiendo en paralelo de una misma cola, la simple llegada de un mensaje de EOF a uno de ellos no alcanza para asegurar que todos los datos del cliente hayan sido efectivamente procesados: aún pueden existir mensajes en tránsito, en *buffers* internos o en proceso de ser *ackeados*. Adoptar una política optimista en este punto comprometería la corrección de los resultados, lo cual es inadmisibles para una aplicación cuyo dominio es la detección de patrones en transacciones financieras.

5.1. Topología de anillo

Para resolver este problema se adoptó una topología de anillo entre los *workers* de cada grupo. Cuando mencionamos que un conjunto de N *workers* idénticos utiliza topología de anillo para el procesamiento de la señal EOF, cada *worker* i tiene una cola privada `worker_ringi` de la que consume y publica a la cola `worker_ring-(i+1) mod N` del siguiente. El EOF que publica el *gateway* viaja junto con los datos: de esa manera, llega a *worker* que el algoritmo de RabbitMQ haya elegido acompañado del total de filas que el *gateway* envió al respectivo tópico. Esa cifra es la *verdad de referencia* contra la cual el anillo decidirá si todos los datos fueron consumidos.

El protocolo se compone de dos rondas que viajan por el anillo:

- **Ronda de colecta.** El *worker* que recibió el EOF se declara *iniciador*, guarda el total esperado y arranca un mensaje con su propio conteo de filas procesadas para ese cliente. El mensaje recorre el anillo, y cada *worker* intermedio simplemente suma su conteo acumulado al valor que viene y reenvía al siguiente. Los *workers* intermedios no necesitan recordar el mensaje después de reenviarlo: su único aporte es agregar su parte a la suma parcial. Cuando el mensaje vuelve al iniciador, éste compara el total acumulado contra el total esperado. Si los números coinciden, dispara la ronda de envío. Si no coinciden, el iniciador republica el mismo EOF sobre la cola de la cual lo recibió, para que eventualmente algún *worker* (puede ser él mismo u otro) reinicie la colecta. La hipótesis subyacente es que, si la cuenta no dio, algún *worker* todavía tenía filas en *buffer* sin procesar, y darles una vuelta más les concede el tiempo necesario para drenar esos *buffers* y converger a la cuenta correcta.
- **Ronda de envío.** El iniciador *flushes* sus datos acumulados a la cola correspondiente junto con un EOF y envía un segundo mensaje por el anillo. Cada *worker* intermedio, al recibirlo, hace lo mismo: *flushes* sus datos acumulados con su propio EOF, borra el estado local de los datos que *flushes* y reenvía. Cuando el mensaje vuelve al iniciador, éste da por finalizada la ronda. El componente posterior espera N EOFs por cliente, los recibe gracias a la combinación de los *flushes* de cada *worker* y, recién entonces, procesa la consulta para ese cliente.

De esta manera, el sistema garantiza que ningún componente posterior procese datos de un cliente hasta que todos los *workers* hayan *flushado* sus aportes para ese cliente, lo cual asegura la consistencia y completitud del resultado.

5.2. Garantía de entrega confiable

La garantía de entrega confiable descansa sobre el hecho de que el EOF llega con el total de filas que el cliente envió (para ese tópico). Mientras ese total no coincida con la suma de lo que reportan los *workers*, el sistema se niega a hacer un flush de los datos parciales y republica el EOF. Cada reintento le concede a los *workers* “lentos” más tiempo para drenar sus *buffers*, y el sistema converge eventualmente al resultado correcto. Esto es especialmente importante para consultas que requieren procesar grandes volúmenes de datos, ya que permite manejar situaciones donde algunos *workers* resultan más lentos que otros sin comprometer la integridad del procesamiento.

5.3. Trade-offs

- **Mensajes del protocolo.** Siendo N la cantidad de *workers* en el anillo, este utiliza $2N$ mensajes por cliente en el caso optimista. La ronda de colecta consume N saltos para que el mensaje vuelva al iniciador, y la ronda de envío otros N saltos. El costo es lineal en la cantidad de *workers* del grupo.
- **Costo del reintento.** En presencia de transacciones en *buffer* cuando arranca la primera colecta, el conteo no cierra y se requiere una vuelta de N mensajes adicionales por reintento. Si un mensaje en vuelo se atrasa demasiado, pueden producirse varias vueltas de reintento, lo cual agrega latencia. Sin embargo, esa latencia es el precio de garantizar la entrega confiable sin perder datos, y resulta un compromiso razonable dado que el sistema converge eventualmente al resultado correcto.
- **Frente a un esquema centralizado.** Una alternativa considerada y descartada fue centralizar el conteo en un nodo coordinador mediante confirmaciones explícitas de cada *worker*. Esa alternativa multiplica la cantidad de mensajes en función del tamaño del *batch* y convierte al nodo coordinador en cuello de botella, además de introducir un punto de falla adicional. El anillo, en cambio, distribuye la responsabilidad y mantiene el costo acotado.

En las secciones siguientes, cuando expliquemos cada consulta Q1–Q5 y aparezcan los anillos de control en los diagramas de robustez, se asume que este protocolo garantiza la coherencia del procesamiento.

6. Vista Física y de Despliegue

En esta sección describimos cómo se materializan los nodos lógicos definidos en el DAG sobre los recursos físicos del sistema. Se presentan, primero, los diagramas de robustez, que muestran las colas, *exchanges* y *workers* concretos involucrados en cada consulta y, posteriormente, el diagrama de despliegue, que sitúa estos componentes sobre los nodos físicos.

6.1. Tipos de nodos lógicos

Los nodos representados en los diagramas se organizan en dos categorías según su responsabilidad dentro del sistema: los **actores**, que participan en la interacción con componentes externos o en la coordinación del flujo principal, y los **workers**, que realizan tareas internas de procesamiento sobre los datos.

6.1.1. Actores

Los actores corresponden a nodos con responsabilidades específicas dentro de la arquitectura, como la recepción de solicitudes del cliente, la comunicación con servicios externos y la obtención de información desde APIs. En esta categoría se incluyen los nodos *Client*, *Gateway* y *Fetcher*.

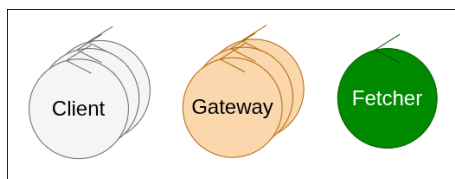


Figura 7: Actores del sistema.

- **Client:** lee los datasets, envía sus transacciones y cuentas al sistema y recibe los resultados consolidados. Es quien inicia y consume cada ejecución.

- **Gateway:** actúa como puerta de entrada y de salida. Desempaqueta los lotes que envía el cliente e inyecta cada transacción o cuenta al flujo interno (con su *ClientId* y *GatewayId*); en sentido inverso, recolecta los resultados finales y los devuelve al cliente que los solicitó.
- **Fetcher:** obtiene de un servicio externo los tipos de cambio del período de interés y los pone a disposición del pipeline, de modo que las consultas puedan convertir los montos a USD (lo requiere Q5).

6.1.2. Workers

Todos los workers comparten la misma estructura: consumen tuplas, les aplican una transformación y emiten el resultado, y se diferencian únicamente por la *estrategia* que ejecutan.

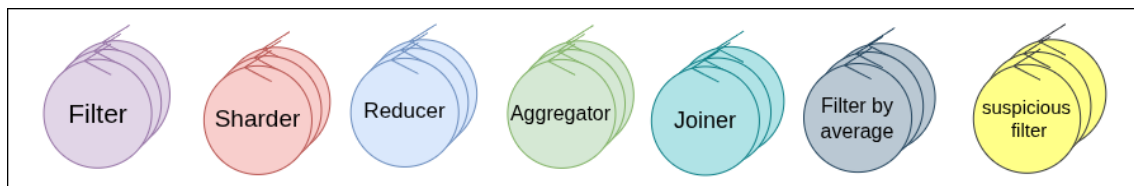


Figura 8: Estrategias de los workers.

- **Filter:** evalúa un predicado sobre cada tupla y la deja pasar o la descarta (por moneda, período, monto o medio de pago).
- **Sharder:** redistribuye las tuplas según una clave (cuenta o banco) para que la etapa siguiente pueda procesarlas.
- **Reducer:** calcula un resultado parcial sobre el subconjunto de tuplas que le toca (un máximo, una suma con su conteo o los caminos entre cuentas).
- **Aggregator:** combina los resultados parciales de los *reducers* en el resultado global de la rama (el máximo global, el promedio o el conteo total).
- **Joiner:** une dos flujos relacionados en uno solo; por ejemplo, cruza el máximo de cada banco con su nombre, o consolida respuestas de un cliente.
- **Filter by average:** variante del filtro propia de Q3 que conserva únicamente las transacciones cuyo monto es inferior a la centésima parte del promedio previamente calculado.
- **Suspicious filter:** pre-filtro propio de Q4 que conserva únicamente las cuentas con un mínimo de contrapartes distintas (puntos de entrada o de salida) y descarta el resto antes de reconstruir los caminos.

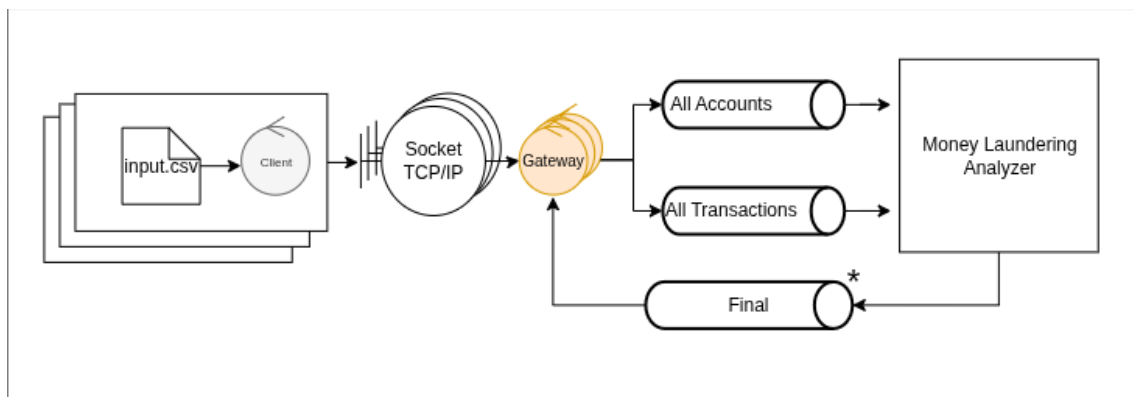
Es importante aclarar que los nodos de entrada de cada consulta (sean Sharder / Filter o Reducers) son los encargados de la coordinación del EOF mediante la topología de anillo (descrito en la sección *Manejo de EOF*) y de aplicar la clave de sharding correspondiente para derivar los datos a las etapas posteriores.

6.2. Diagrama de Robustez

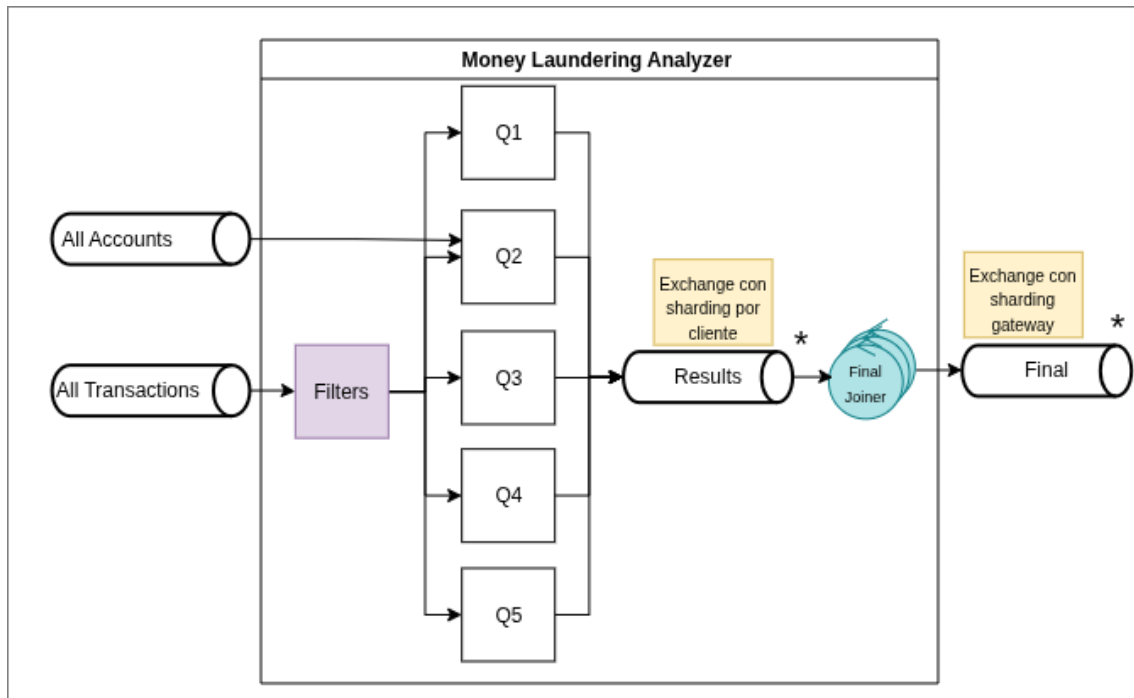
A continuación se presenta la vista general del sistema y, posteriormente, el detalle de cada consulta Q1–Q5. La nomenclatura es uniforme en todos los diagramas: los cilindros representan colas o *exchanges* de RabbitMQ, los controladores encerrados en un cuadrado punteado en fucsia representan grupos de *workers* idénticos coordinados mediante topología de anillo (cuyo protocolo se describió en la sección anterior).

Por su parte, el asterisco (*) sobre una cola indica que, en realidad, representa un exchange direct con múltiples colas, una por cada instancia del worker siguiente. Estas colas son alimentadas por un worker previo al Exchange que se encarga de aplicar una función de hash (sharding) sobre una clave específica (por ejemplo, el ID del cliente, el formato de pago o la cuenta bancaria). Esto garantiza que todos los eventos asociados a una misma entidad sean enrutados y encolados siempre en la misma partición, permitiendo mantener un estado consistente en memoria sin necesidad de locks distribuidos. Por otro lado, la estrella (★) representa que se trata de un exchange fanout.

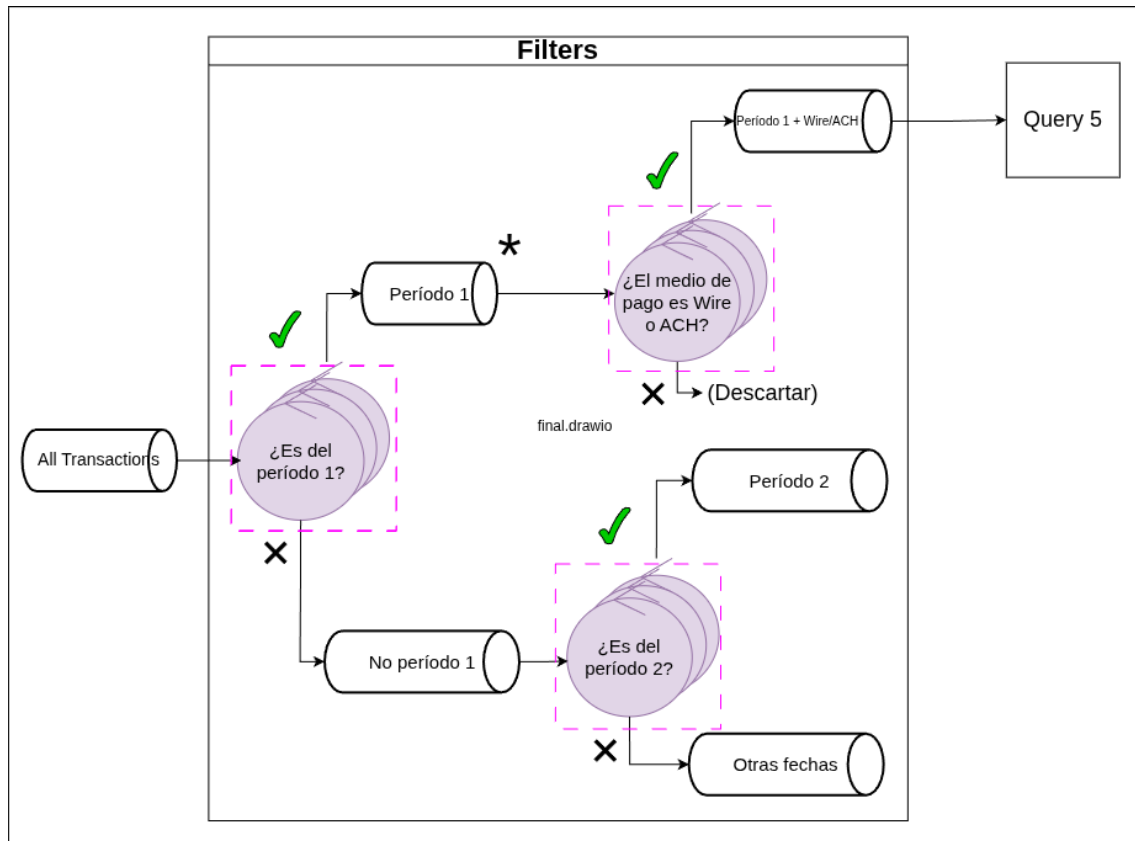
- **Vista general.** El cliente se conecta al *Gateway* mediante un socket TCP/IP, y envía las transacciones y las cuentas bancarias. El *gateway* encola estos datos en las colas *All Transactions* y *All Accounts* respectivamente. El sistema procesa los datos, genera los resultados de cada consulta y los encola en *Final* shardeado por cliente, de forma que el *Gateway* del mismo cliente los reciba y se los devuelva.

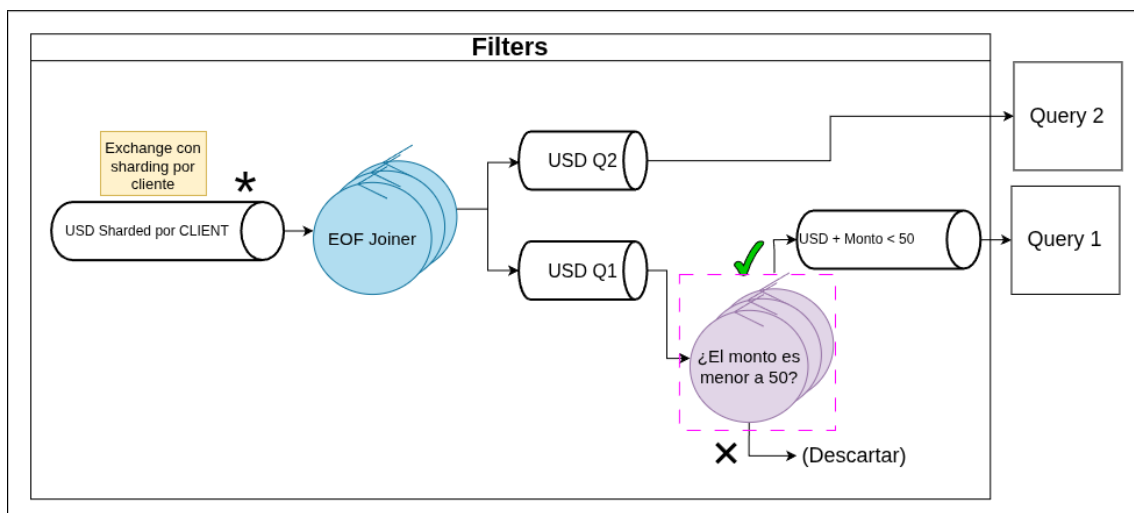
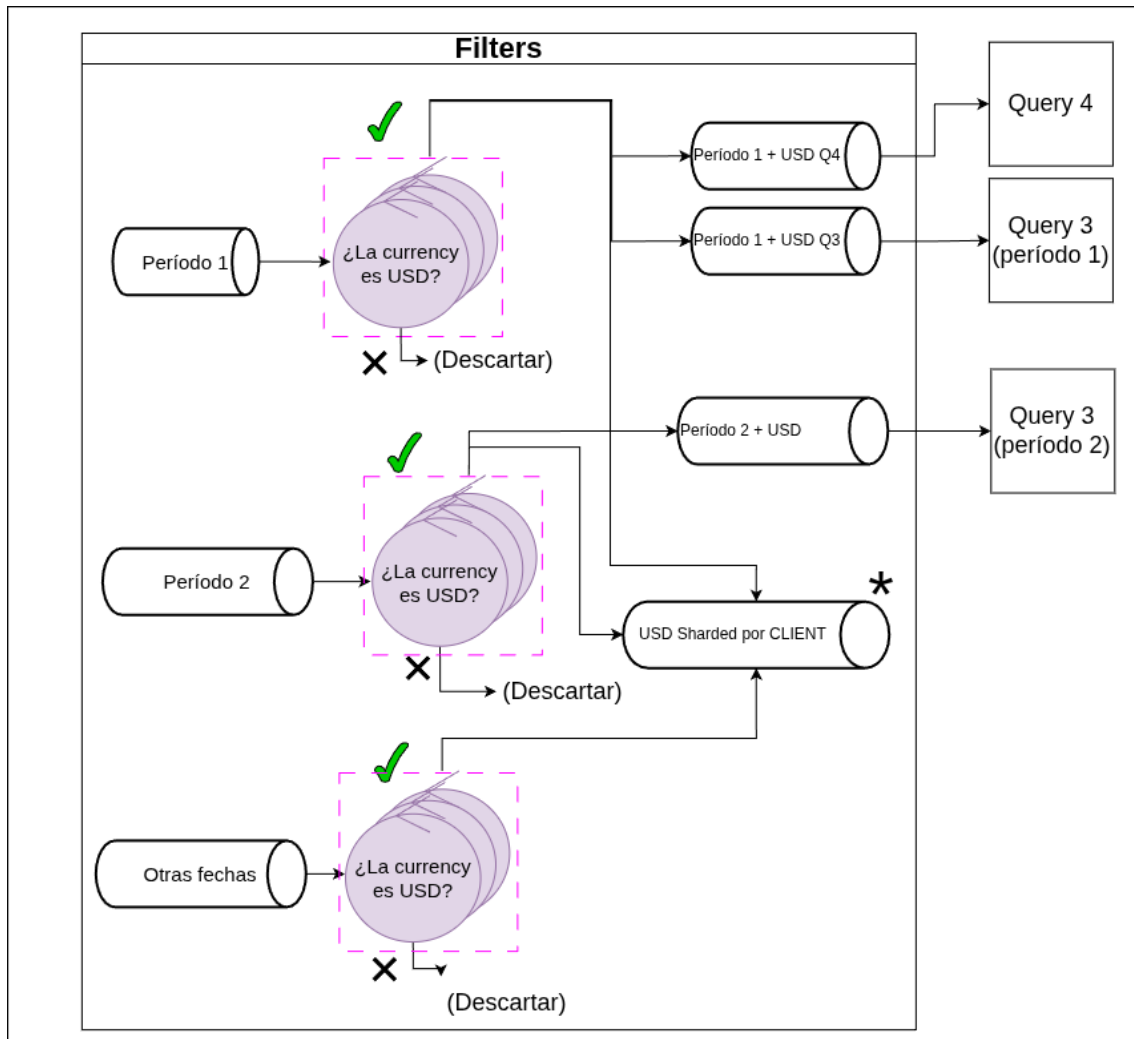


- **Vista general de los nodos de procesamiento.** Un conjunto de nodos *Filter* se encarga de filtrar y descartar columnas de las transacciones de acuerdo a lo necesitado por cada una de las consultas. Los datos llegan mediante colas a cada uno de los grupos de nodos de las consultas (Q1 a Q5). El *Final Joiner* recibe los resultados de todas las consultas desde el exchange *Results* y los rutea a la cola del gateway correspondiente a cada cliente. Además, consolida los múltiples EOF que emiten las réplicas de cada query en un único EOF por consulta, señalizando al gateway que esa query finalizó para ese cliente.



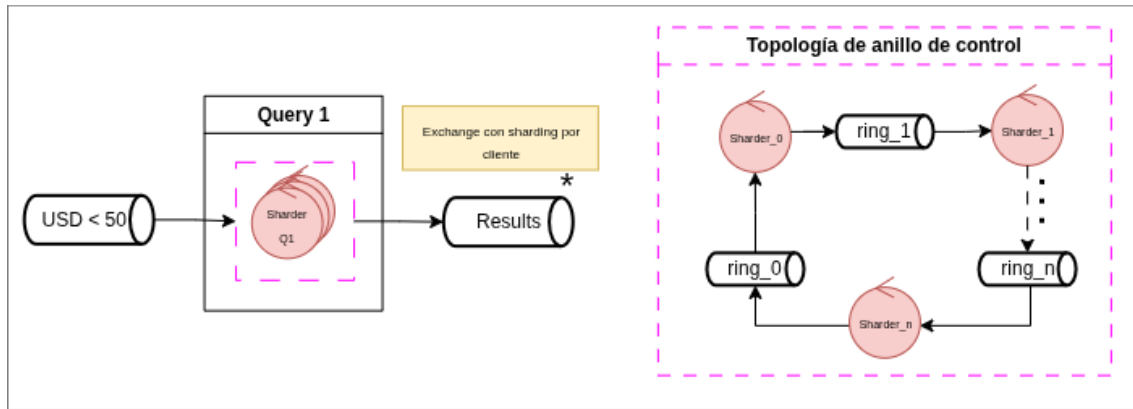
- **Filtros.** Cada grupo de nodos filtro recibe transacciones de una única cola o exchange, y posee dos conjuntos de middlewares de salida. Para cada una de las transacciones, evalúa un predicado y según el resultado (booleano) la encola en los middlewares de un conjunto de salida u del otro. En algunos grupos de *Filters*, el conjunto de salida correspondiente al no cumplimiento del predicado está vacío, por lo que las transacciones que llegan hasta ahí se descartan. Los últimos grupos de *Filters* encolan las transacciones en los mismos middlewares que consumen los nodos de procesamiento de cada consulta. Debido a que las consultas tienen condiciones de filtrado que son compartidas, la topología de nodos *Filter* no es puramente la de un árbol, sino que en un punto se vuelven a juntar flujos separados (más específicamente, en el exchange *USD Sharded por client*, por donde pasan todas las transacciones en dólares). Esta particularidad requiere agregar un grupo de nodos *EOF Joiner*, los cuales se encargan de unir los tres mensajes de EOF provenientes de los flujos convergentes, y generar un único mensaje combinado con la suma de los tres. Para facilitar la lectura, se dividió el diagrama de robustez de esta parte del sistema en tres:





- **Query 1 (Micropagos).** Por su simplicidad, esta consulta se resuelve con un único grupo de *workers* idénticos (*Sharders*) en topología de anillo. Cada *Sharder* consume de la misma cola (con transacciones ya filtradas), proyecta la tupla y la encola en **Results**. El recuadro a la derecha del diagrama explicita la topología del anillo de control: cada *Sharder* i consume de su cola privada $ring_i$ y publica a $ring_i \mod n$ para coordinar el cierre por EOF

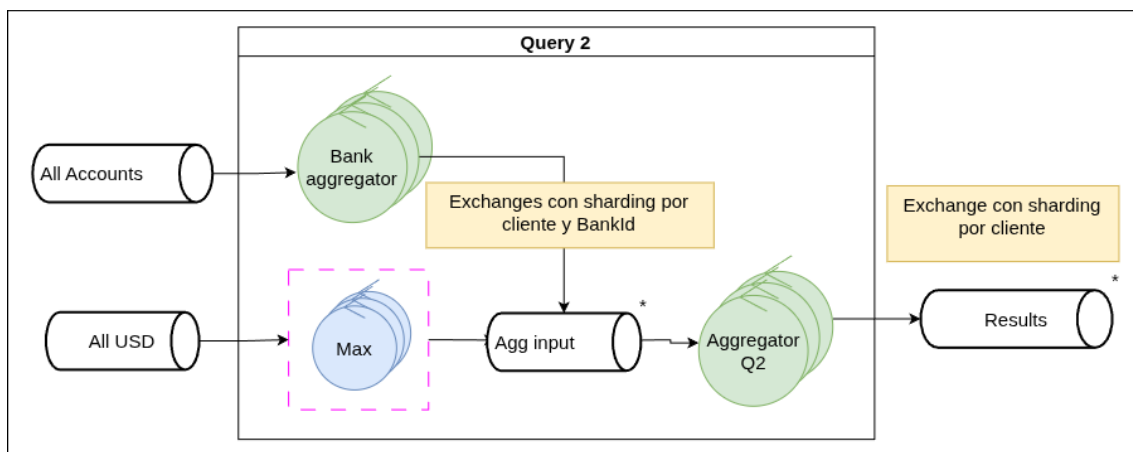
descrito previamente.



- **Query 2 (Máximos por Entidad).** La consulta Q2 implementa un esquema de reducción local seguido de una etapa de agregación con *join*. El procesamiento se divide en dos ramas que operan en paralelo. Por un lado, los nodos *Max.i* consumen las transacciones en USD y mantienen, para cada banco, el monto máximo observado localmente. Al finalizar el procesamiento del cliente, envían sus resultados parciales al *exchange q2_agg_input*.

En paralelo, los nodos *Bank Aggregator.i* procesan el archivo de cuentas y construyen un diccionario que asocia cada *bankID* con el nombre correspondiente del banco. Estos parciales también son enviados al mismo *exchange* una vez recibido el EOF.

Ambas ramas distribuyen sus mensajes utilizando *sharding* por (*clientID*, *bankID*), asegurando que un mismo nodo *Aggregator.i* reciba tanto los máximos calculados como la información descriptiva del banco asociado. Cuando el agregador recibe todos los EOFs esperados —tres provenientes de los nodos *Max* y tres de los *Bank Aggregator*— realiza el *join* entre ambas fuentes, reemplaza los identificadores por los nombres de banco correspondientes y publica el resultado final en *Results*.



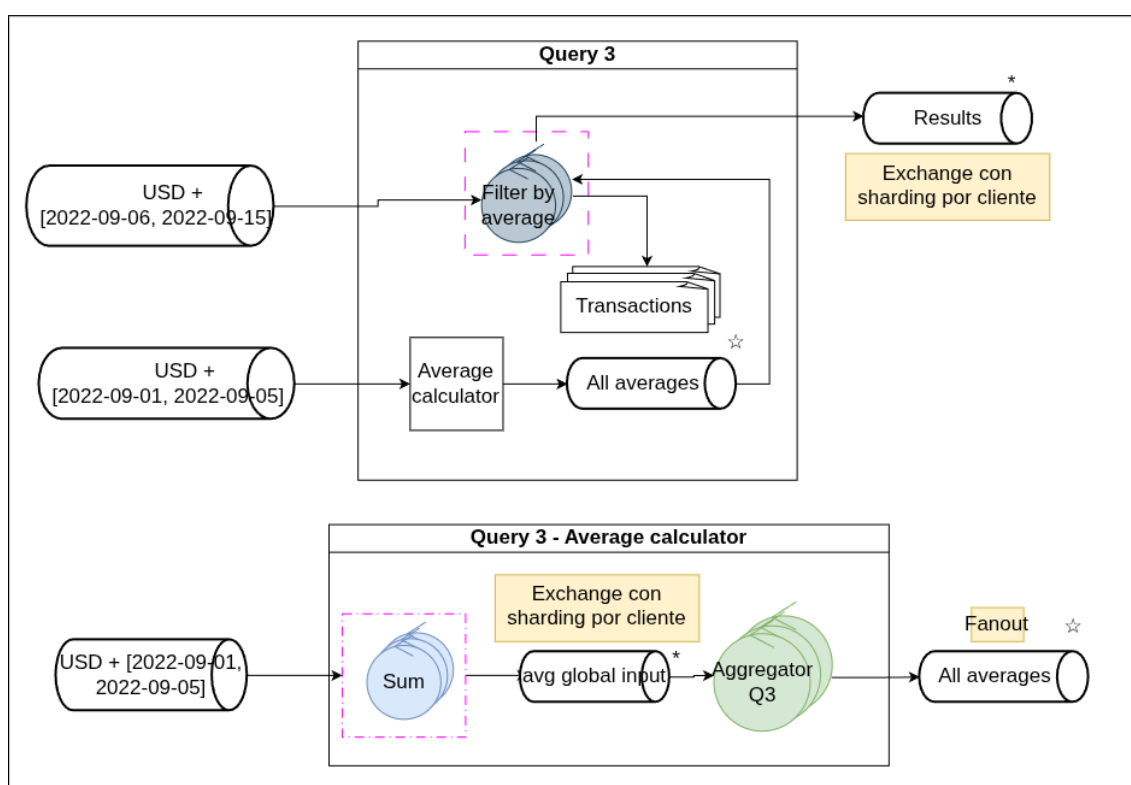
- **Query 3 (Anomalías Históricas).** Esta consulta se organiza en dos sub-ramas que convergen y se sincronizan en el componente *Filter By Average*.

La primera sub-rama tiene como objetivo calcular el promedio global por formato de pago. Para ello, los nodos *Sum.i* —un conjunto de *workers* idénticos coordinados en anillo— consumen las transacciones en USD correspondientes al período 1 y mantienen, para cada formato de pago, una suma acumulada y un conteo parcial. Al finalizar el procesamiento del cliente, emiten dichos parciales al *exchange Partial sums*, utilizando *sharding* por cliente. Luego, cada *Aggregator Q3.i* consume los parciales asignados, calcula el promedio final por

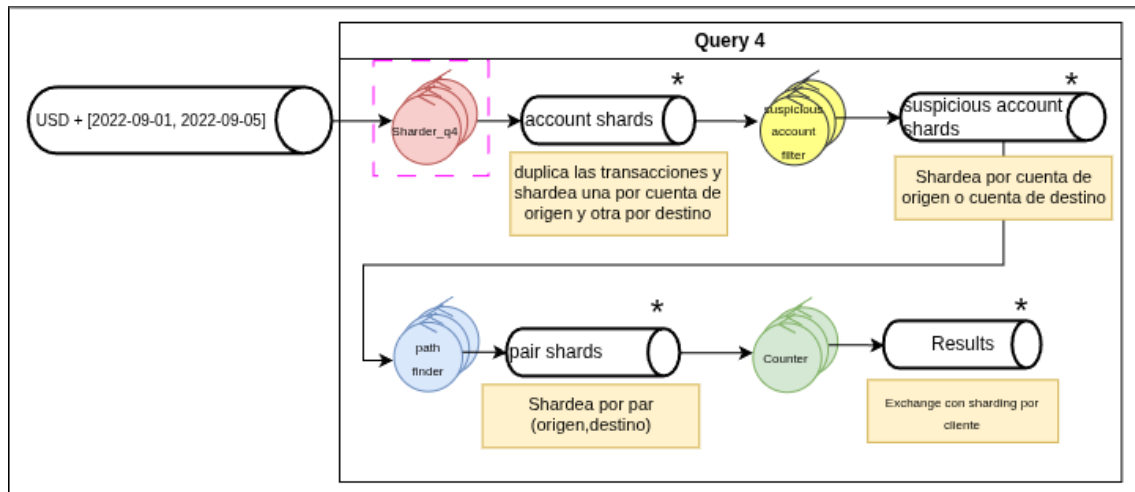
formato de pago y difunde esos resultados a todos los nodos *Filter By Average* mediante un mecanismo de *broadcast*.

La segunda sub-rama se encarga de filtrar las transacciones correspondientes al período 2. Cada *Filter By Average* consume simultáneamente dos flujos: las transacciones USD y los promedios distribuidos desde el *exchange Averages*. Si las transacciones llegan antes de que los promedios estén completos, estas se persisten temporalmente en disco; en cambio, una vez disponibles los promedios, las nuevas transacciones pueden filtrarse en línea a medida que arriban.

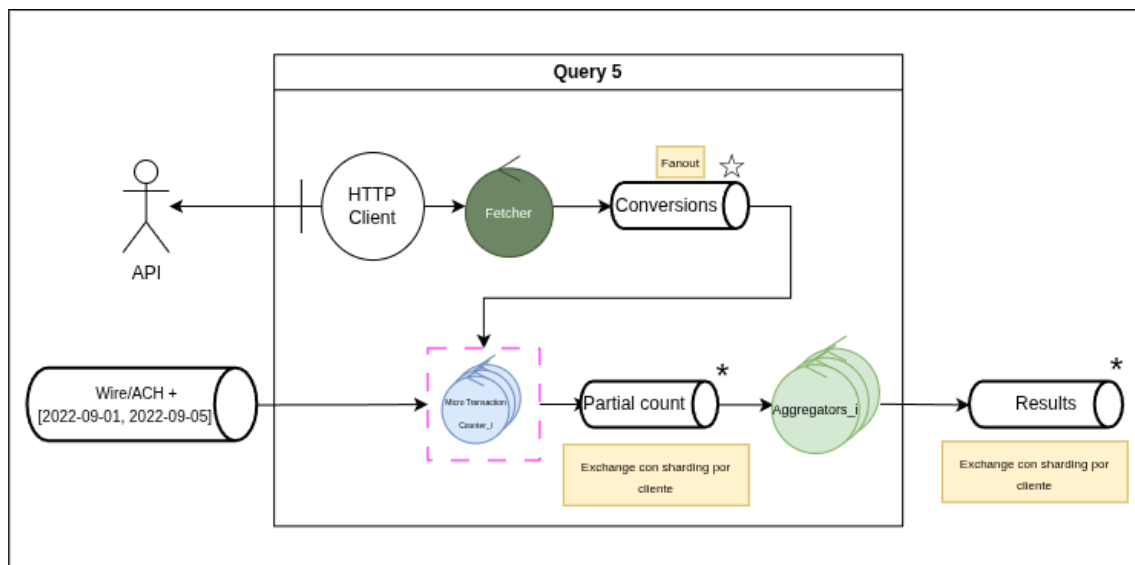
Cuando el componente recibe tanto el EOF del flujo de transacciones como el EOF asociado a los promedios, recorre el buffer persistido en disco y conserva únicamente aquellas transacciones cuyo monto resulta inferior al 1 % del promedio correspondiente a su formato de pago. Finalmente, las transacciones filtradas son emitidas hacia **Results**.



- Query 4 (Scatter-Gather).** La consulta está compuesta por cuatro etapas en *pipeline*. El *Sharder_i* consume las transacciones USD del período 1 y emite cada una duplicada al *exchange Account Shards*: una copia *shardeada* por cuenta de origen y otra por cuenta de destino. El *Suspicious Account Filter_i* consume de su shard de *Account Shards* y detecta cuentas con cinco o más contrapartes distintas; al superarse el umbral, emite todas las transacciones acumuladas de esa cuenta ahora *shardeadas* por la contraparte al *exchange Suspicious Account Shards*. El *Path Finder_i* consume de *Suspicious Account Shards* y reconstruye caminos de tres saltos $A \rightarrow M \rightarrow B$ a través de cuentas intermediarias sospechosas, emitiendo cada conjunto al *exchange Pair Shards* *shardeado* por el par (A, B). Finalmente, el *Counter_i* acumula las cuentas intermediarias distintas por par (A, B) y emite a **Results** aquellos pares que superen las cinco intermediarias. El detalle del algoritmo del *Path Finder* y un ejemplo de ejecución se presentan en la *Vista de Procesos*.



- **Query 5 (Micropagos Internacionales).** La consulta incorpora una particularidad respecto a las anteriores: depende de un servicio externo. El nodo único *Fetcher* consulta, vía cliente HTTP, la API de Frankfurter para obtener y enviar via fanout las cotizaciones diarias necesarias para la conversión a USD. El grupo de nodos *Micro Transaction counter* desencola esos valores hasta asegurarse de haber obtenido todos. Una vez hecho esto cambia su cola de input y desencola transacciones convirtiendo su monto a dólares. Se incrementa un contador parcial sólo si el monto resulta menor a 1 USD. Al cierre del cliente, los parciales se emiten al *exchange Partial count*, y el *Aggregator_i* consolida la suma final por cliente y la publica en *Results*.



Decidimos realizar una única consulta a la API al iniciar el nodo *Fetcher* y almacenar los resultados en memoria en cada *Converter*. Ya que pudimos observar que es factible almacenar todos los resultados en memoria, dado que los volúmenes de datos son razonables. Por ejemplo, una consulta para la conversión de todas las monedas a USD durante un período de un año ocupa aproximadamente 3.5 MB, mientras que para el período solicitado en el trabajo práctico el tamaño es de alrededor de 40 kB.

Como se observa:

```
1 $ curl -H "Accept: application/x-ndjson" "https://api.frankfurter.dev/v2/rates?from=2022-01-01&to=2022-12-31&base=USD" -o
   rates_2022.ndjson
2 % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
3    Dload  Upload   Total   Spent    Left   Speed
4 100 3067k    0 3067k    0    0    275k    0  --:--:--  0:00:11  --:--:-- 359k
```

Listing 1: Estadísticas de la solicitud a la API para un período de un año entero

```

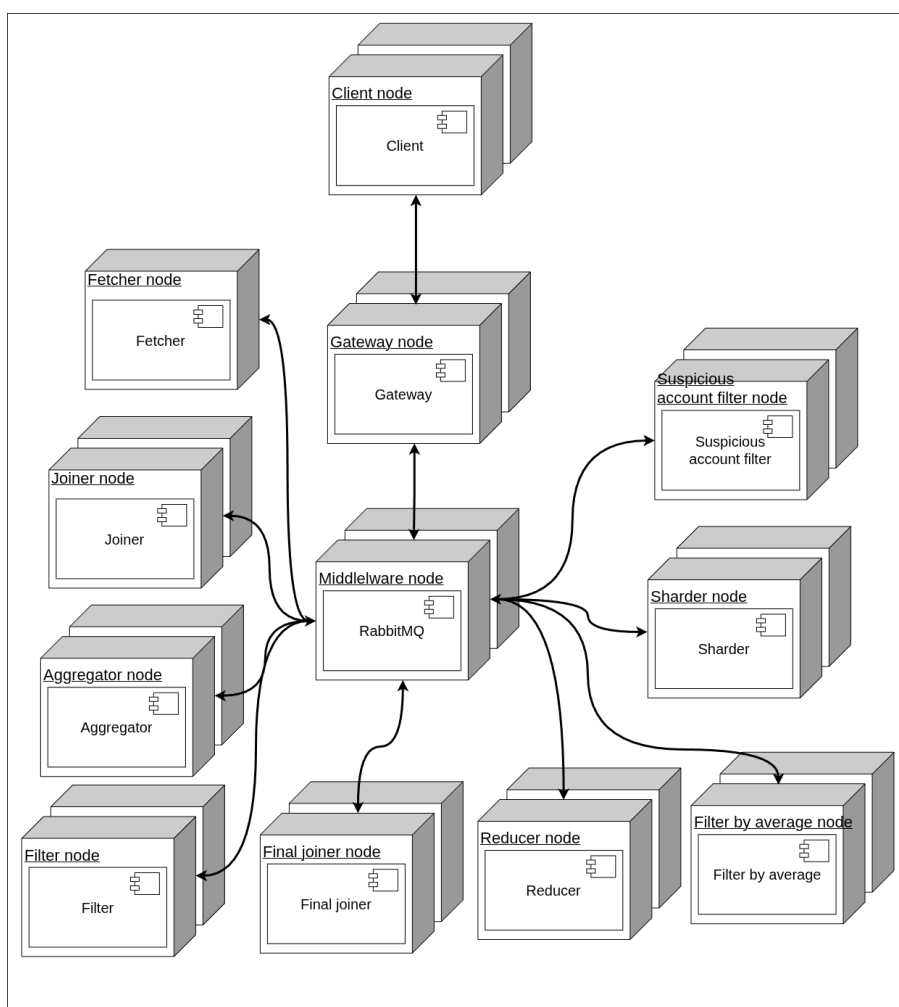
1 $ curl -H "Accept: application/x-ndjson" "https://api.frankfurter.dev/v2/rates?from=2022-09-01&to=2022-09-05&base=USD" -o
   rates_period.ndjson
2 % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
3           Dload  Upload   Total   Spent    Left     Speed
4 100 39708    0 39708    0    0 32261    0 --:--:--  0:00:01 --:--:-- 32256

```

Listing 2: Estadísticas de la solicitud a la API para el período solicitado en el trabajo práctico

6.3. Diagrama de Despliegue

El siguiente diagrama presenta la organización física de los componentes sobre nodos de cómputo. Cada nodo se ejecuta en un contenedor Docker independiente y toda la comunicación entre ellos atraviesa el nodo de *middleware*, que aloja el *broker* RabbitMQ. Esta indirección provee la transparencia que requiere un sistema basado en grupos, donde los *workers* no se conocen entre sí, sino que se acoplan únicamente a las colas y *exchanges* sobre los que producen o consumen.



Los paquetes representados en el diagrama corresponden a las categorías definidas en la sección *Tipos de nodos*. Los **actores** se ubican en los extremos de la arquitectura y son responsables de la interacción con componentes externos. El nodo *Client* establece comunicación con el *Gateway* mediante sockets TCP/IP, mientras que el *Gateway* actúa como punto de enlace entre los clientes y el *middleware*, encargándose tanto de recibir y desempaquetar los lotes de mensajes como de retornar los resultados procesados. Por su parte, el nodo *Fetcher* obtiene información desde APIs externas y publica las cotizaciones en RabbitMQ.

El resto de los componentes corresponde a los **workers** (*Filter*, *Sharder*, *Reducer*, *Aggregator*, *Joiner*, *Final joiner*, *Filter by average* y *Suspicious account filter*), cada uno desplegado en un

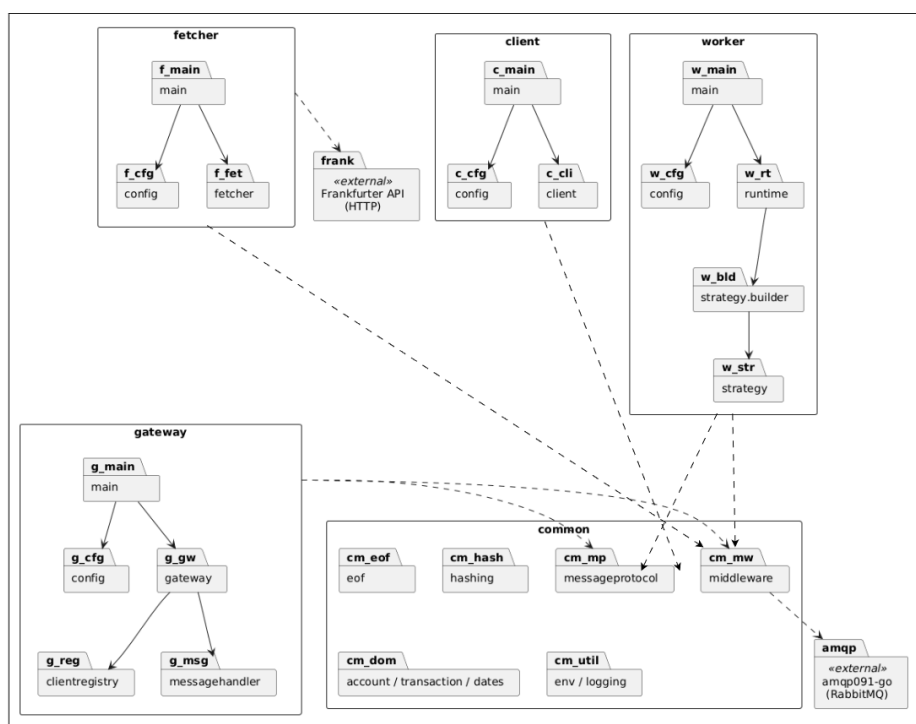
nodos independientes y comunicados exclusivamente a través del *middleware*. Cada grupo de *workers* se escala horizontalmente de forma independiente, replicando su contenedor y sumándolo si corresponde al anillo de control de su grupo (descrito en *Manejo de EOF*). Por simplicidad gráfica, los nodos de un mismo tipo se agrupan aunque no ejecuten exactamente la misma tarea en cada consulta (por ejemplo, los *aggregators* de Q2, Q3 y Q5 realizan estrategias de agregación distintas).

7. Vista de Desarrollo

En esta sección presentamos la organización del código fuente del sistema, agrupado en paquetes que se corresponden con las unidades de despliegue y con los componentes transversales reutilizados por más de una unidad.

7.1. Diagrama de Paquetes

El código se estructura en cuatro unidades desplegables — *client*, *gateway*, *worker* y *fetcher* — que se apoyan en una biblioteca compartida, *common*. El diagrama se derivó directamente del grafo de *imports* del módulo, a nivel de paquete, y se presenta en dos partes: la primera con *client*, *worker* y *fetcher*; la segunda con *gateway* y *common*. Las unidades de la primera figura dependen de *common* (segunda figura) según se detalla a continuación.



- **client.** Su *main* delega en el paquete *config* y *client*, que lee los datasets de entrada, envía los *batches* al *gateway* y recibe los resultados consolidados. Usa de *common* los paquetes *account*, *transaction* y *messageprotocol/external* (protocolo TCP con el *gateway*).
- **gateway.** Su *main* delega en el paquete *gateway*, que coordina *config*, *clientregistry* (asociación clientesocket) y *messagehandler* (desempaqueta los *batches* entrantes y publica cada transacción o cuenta al *middleware*). Usa de *common* *messageprotocol* (externo e interno), *middleware*, *account* y *transaction*.

- **worker.** Su *main* delega en *runtime*, el bucle genérico (consumir → transformar → emitir) que despacha hacia *strategy* y su *builder*. *strategy* concentra la lógica concreta de cada Q1–Q5 y de los auxiliares (*sharder*, *suspicious filter*, etc.), siguiendo el patrón *Strategy*. Usa de *common eof*, *hashing*, *messageprotocol* (interno y externo) y *middleware*.
- **fetcher.** Su *main* delega en el paquete *fetcher*, que consulta por HTTP la API de Frankfurter para obtener las cotizaciones del período y las publica al *middleware* para que Q5 convierta los montos a USD.
- **common.** Biblioteca transversal reutilizada por todas las unidades. Agrupa *messageprotocol* —con *external* (safeio para el socket y *serializer* para el formato del protocolo TCP clientegateway) e *inner* (mensajes internos sobre RabbitMQ)—, *middleware* (abstrae AMQP), *eof* (protocolo de cierre en anillo), *hashing* (*sharding*) y los paquetes de dominio y utilidad *account*, *transaction*, *dates*, *env* y *logging*.

Concentrar lo compartido en *common* evita duplicar código entre unidades y aísla las decisiones tecnológicas en un único lugar. De esta forma, RabbitMQ queda contenido en *common/middleware* y el cliente HTTP, en el *fetcher*.

8. Planificación y Organización

En la presente sección consignamos el listado de tareas que componen el desarrollo del sistema, junto con la división tentativa entre los integrantes del equipo. La granularidad de las tareas se eligió de modo de coincidir, en líneas generales, con los componentes identificados en las vistas anteriores, lo que facilitará la integración y reducirá las dependencias entre integrantes durante la implementación.

8.1. Listado de Tareas a Ejecutar

- **Cliente.** Implementación del lector de CSV, armado y envío de *batches* por TCP, manejo de la señal de EOF hacia el *gateway* y recepción de los resultados consolidados.
- **Gateway + Tagger.** Implementación del servidor TCP, asociación de *ClientID* a sockets, lógica de *tagging* por contenido de transacción, contadores por *topic* para inyectar el total esperado en los EOFs y reenvío de los resultados consolidados al cliente correspondiente.
- **Protocolo cliente ↔ gateway.** Definición del formato de *batches*, señal de EOF y mensajes de respuesta. Implementación dentro del paquete transversal *generic.communication*.
- **Protocolo de anillo de EOF.** Implementación de las dos rondas (colecta y envío) descritas en la sección *Manejo de EOF*, así como el mecanismo de reintento ante discrepancia con el total esperado.
- **Workers de Q1.** Implementación del *Dispatcher* con anillo de control y emisión a *Results*.
- **Workers de Q2.** Implementación del *Max* local con anillo de control y del *Aggregator* con *sharding* por cliente.
- **Workers de Q3.** Implementación de las dos sub-ramas: *Sum* con anillo de control, *Aggregator* y *Joiner* para el *Average Calculator*; y *Period filter* con persistencia local de transacciones a la espera del promedio.
- **Workers de Q4.** Implementación del *Sharder*, del *3-Path Finder* (con su estructura interna de cuentas *Entrada/Salida*) y del *Intermediate Account Counter* con *sharding* por par (origen, destino).

- **Workers de Q5.** Implementación del cliente HTTP hacia la API de Frankfurter (paquete `external_apis`), del *Converter* con anillo de control y del *Aggregator* con *sharding* por cliente.
- **Final Joiner.** Implementación del consumidor del *exchange Results* con *sharding* por cliente, gestión del estado parcial por cliente y emisión de la respuesta consolidada a *Final* cuando se reciben los EOFs de las cinco ramas.
- **Despliegue.** Construcción de las imágenes Docker de cada unidad (*client*, *gateway*, *worker*), *compose* de la topología completa con RabbitMQ y configuración de las réplicas iniciales por grupo.
- **Pruebas funcionales.** Verificación de los resultados de cada consulta contra el *notebook* patrón provisto por la cátedra.

8.2. División del Trabajo entre Integrantes

Tarea	Encargado
Cliente	Juan Martín de la Cruz
Gateway + Tagger	Ian Klaus von der Heyde
Protocolo cliente ↔ gateway	Ian Klaus von der Heyde
Protocolo de anillo de EOF	Ian Klaus von der Heyde
Workers de Q1 (Micropagos)	Juan Martín de la Cruz
Workers de Q2 (Máximos)	Juan Martín de la Cruz
Workers de Q3 (Promedios)	Agustín Altamirano
Workers de Q4 (Scatter-Gather)	Agustín Altamirano
Workers de Q5 (Cotizaciones API)	Ian Klaus von der Heyde
Final Joiner	Juan Martín de la Cruz
Despliegue (Docker/Compose)	Agustín Altamirano, Ian Klaus von der Heyde, Juan Martín de la Cruz
Pruebas funcionales	Ian Klaus von der Heyde